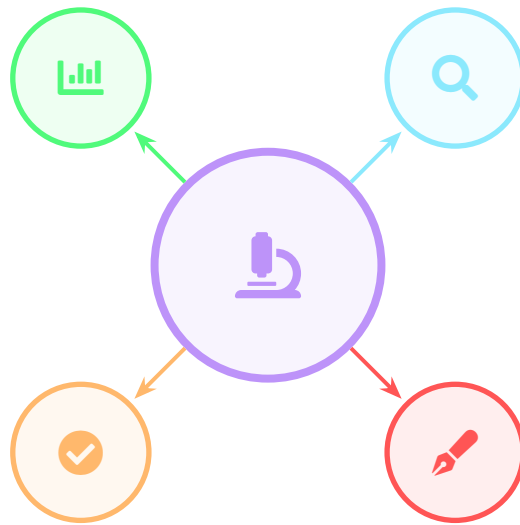




Multi-Agent Research Orchestrator

Complete Hinglish Master Notes

65+ Pages • Line-by-Line Code Explanation • 50 Interview Q&A
RAG • MCP • LangGraph • Groq • ChromaDB • Streamlit

Tech Stack at a Glance

 LLM	Groq (LLaMA 3.3 70B Versatile)
 Orchestration	LangGraph StateGraph
 Web Search	Tavily (Advanced Depth)
 Vector Store	ChromaDB (Persistent)
 Embeddings	HuggingFace all-MiniLM-L6-v2 (Free, Local)
 UI	Streamlit (Glassmorphism Dark Theme)
 Protocol	MCP (Model Context Protocol) via FastMCP
 Language	Python 3.10+

August 11, 2026

Contents

1	Project Overview — Kahani Ki Shuruaat	3
1.1	Project Architecture Diagram	3
1.2	Folder Structure	3
1.3	Dependencies Explained (requirements.txt)	4
2	Configuration — config/settings.py	6
2.1	Complete Code with Line-by-Line Explanation	6
3	State Schema — state/schema.py	9
3.1	Complete Code with Explanation	9
3.2	State Fields Ka Role Table	11
4	Researcher Agent — agents/researcher.py	13
4.1	Complete Code with Detailed Commentary	13
5	Analyst Agent — agents/analyst.py	18
5.1	Complete Code with Commentary	18
6	Fact Checker Agent — agents/fact_checker.py	21
6.1	Complete Code	21
7	Writer Agent — agents/writer.py	25
7.1	Complete Code	25
8	LangGraph Workflow — graph/workflow.py	30
8.1	Complete Code	30
9	RAG System — rag/vector_store.py	34
9.1	Embedding Mathematics	34
9.2	Chunking Strategy	35
9.3	Complete Code	35
10	MCP Server — mcp_server.py	42
10.1	MCP Architecture	42
10.2	Complete Code	42
11	Streamlit UI — app.py Deep Dive	48
11.1	Page Configuration	48
11.2	Premium CSS Design	48
11.3	Session State Management	49
11.4	Pipeline Execution with Streaming	50
11.5	RAG Document Upload UI	52
12	Environment Setup — .env.example	54
12.1	Running the Project	54
13	Advanced Concepts — Deep Technical Dive	56

13.1	LangGraph State Machine Theory	56
13.2	Vector Database Internals	56
13.3	Token Economics	57
13.4	Error Handling Patterns	57
14	Interview Preparation — 50 Questions with In-Depth Answers	59
14.1	Architecture & Design (Q1–Q10)	59
14.2	LangGraph & Orchestration (Q11–Q20)	63
14.3	RAG & Vector Databases (Q21–Q30)	66
14.4	MCP & Integration (Q31–Q37)	69
14.5	Production & Optimization (Q38–Q50)	71
15	Quick Reference Cheat Sheets	76
15.1	LangGraph Cheat Sheet	76
15.2	RAG Cheat Sheet	76
15.3	MCP Cheat Sheet	77
15.4	Streamlit Cheat Sheet	78
16	Glossary — Important Terms	80
17	Final Summary — Pura Project Ek Nazar Mein	82
18	Chapter 11: Auth-First Django REST Framework Backend, Port 8080 & System Architecture	84
18.1	11.1 Backend Architecture Overview: Why Django DRF?	84
18.1.1	Key Concepts:	84
18.2	11.2 Database Schema & Models (<code>backend/api/models.py</code>)	86
18.2.1	Code Explanation:	87
18.3	11.3 Serializers Layer (<code>backend/api/serializers.py</code>)	88
18.3.1	Code Explanation:	89
18.4	11.4 Service Layer Abstraction (<code>backend/api/services.py</code>)	90
18.5	11.5 API Endpoints & Token Authentication (<code>backend/api/views.py</code>)	91
18.6	11.6 Docker Containerization & Deployment	93
18.7	11.7 Backend & Django DRF Interview Questions (Q51 – Q65)	94

1 Project Overview — Kahani Ki Shuru-aat

📖 Kahani Se Samjho

Socho tumhe ek aisa system banana hai jo kisi bhi topic par research kar sake — jaise ek team of 4 experts kaam karti hai:

1. **Researcher (Khoji):** Internet par search karta hai, data collect karta hai. Jaise ek journalist jo field mein jata hai.
2. **Analyst (Vishleshan-karta):** Collected data ko analyze karta hai — patterns dhundhta hai, contradictions nikalta hai. Jaise ek detective jo clues jodta hai.
3. **Fact Checker (Satya-parikshak):** Claims verify karta hai — kya sab sach hai? Agar kuch galat hai, toh wapas Researcher ko bhejta hai. Jaise ek editor jo facts check karta hai.
4. **Writer (Lekhak):** Sab verified data ko ek polished report mein likhta hai with citations. Jaise ek professional writer.

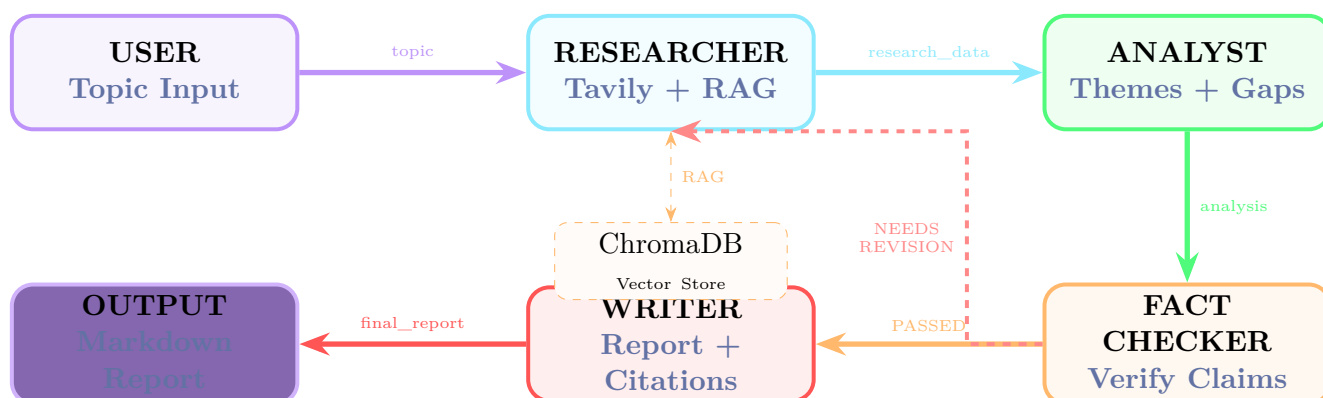
Ab ye 4 agents ek pipeline mein kaam karte hain:

Researcher → Analyst → Fact-Checker → Writer

Aur ek **feedback loop** hai — agar Fact-Checker ko kuch galat milta hai, toh wapas Researcher ko bhejta hai (max 2 baar).

Is project mein humne **RAG (Retrieval Augmented Generation)** bhi add kiya hai — matlab tum apne PDF documents upload kar sakte ho aur agents unhe bhi search karenge. Plus **MCP (Model Context Protocol)** se ye system kisi bhi AI tool (Claude Desktop, Cursor) se connect ho sakta hai!

1.1 Project Architecture Diagram



1.2 Folder Structure

Project ka folder structure samajhna bahut zaroori hai. Ye ek clean, modular structure hai:

```

Multi-Agent-Research-Orchestrator/
|-- app.py                # Streamlit UI (707 lines)
|-- mcp_server.py         # MCP Protocol Server (208 lines)
|-- requirements.txt      # Dependencies
|-- .env.example          # API keys template
|-- .gitignore
|--
|-- config/
|   |-- __init__.py
|   |-- settings.py       # Centralized configuration (42 lines)
|--
|-- state/
|   |-- __init__.py
|   |-- schema.py         # Shared state TypedDict (59 lines)
|--
|-- agents/
|   |-- __init__.py
|   |-- researcher.py     # Web Search + RAG Agent (121 lines)
|   |-- analyst.py        # Analysis Agent (85 lines)
|   |-- fact_checker.py   # Verification Agent (107 lines)
|   |-- writer.py         # Report Writer Agent (135 lines)
|--
|-- graph/
|   |-- __init__.py
|   |-- workflow.py       # LangGraph Pipeline (78 lines)
|--
|-- rag/
|   |-- __init__.py
|   |-- vector_store.py   # ChromaDB RAG Manager (276 lines)

```

💡 Yaad Rakhne Ka Tip

Yaad Rakho: Har folder mein `__init__.py` file hoti hai — ye Python ko batati hai ki “ye ek package hai, isko import kar sakte ho.” Bina iske `from agents.researcher import ...` kaam nahi karega!

1.3 Dependencies Explained (requirements.txt)

Pehle samjho hum kya install kar rahe hain aur **kyun**:

```

1 # Core LangChain Framework
2 langchain>=0.3.0           # Base framework for LLM apps
3 langchain-groq>=0.3.0      # Groq LLM integration
4 langchain-tavily>=0.1.0    # Tavily search tool wrapper
5 langchain-community>=0.3.0 # Community integrations
6 langgraph>=0.4.0           # StateGraph for multi-agent
7 streamlit>=1.40.0          # Web UI framework
8 python-dotenv>=1.0.0       # .env file loader
9 pydantic>=2.0.0            # Data validation

```

```
10
11 # RAG Dependencies
12 langchain-chroma>=0.2.0      # ChromaDB vector store
13 langchain-huggingface>=0.1.0 # HuggingFace embeddings
14 langchain-text-splitters>=0.3.0 # Text chunking
15 pypdf>=5.0.0                # PDF file reader
16
17 # MCP Dependencies
18 fastmcp>=2.0.0              # MCP protocol server
```

 requirements.txt

🔗 Code Samjho Line-by-Line

Har dependency ka role:

- **langchain** — Ye “spine” hai puri app ki. LLM calls, prompts, chains — sab iske through.
- **langchain-groq** — Groq ka LLM use karne ke liye. Groq ultra-fast hai (custom hardware pe LLaMA run karta hai).
- **langchain-tavily** — Tavily ek AI-optimized search engine hai. Google API se zyada structured data deta hai.
- **langgraph** — Multi-agent workflow banane ke liye. Ye “graph” based hai — nodes (agents) aur edges (connections).
- **streamlit** — Python se web app banane ka sabse easy tarika. No HTML/JS needed (mostly).
- **langchain-chroma** — ChromaDB vector database. Documents ko vectors mein store karta hai for semantic search.
- **langchain-huggingface** — HuggingFace se embedding models load karne ke liye. Free aur local.
- **pypdf** — PDF files ko text mein convert karne ke liye.
- **fastmcp** — MCP protocol server banane ke liye. Claude Desktop, Cursor jaisi tools se connect.

2 Configuration — config/settings.py

Kahani Se Samjho

Socho tumhare ghar mein ek **control panel** hai — AC ka temperature, lights ka brightness, wifi ka password — sab ek jagah se control hota hai. **settings.py** exactly wahi hai — **saari configuration ek jagah**, taki agar kuch change karna ho toh sirf ek file mein karo. Isko kehte hain “Single Source of Truth”.

2.1 Complete Code with Line-by-Line Explanation

```

1  """
2  Centralized configuration -- loads .env and exposes settings.
3  """
4
5  import os                                # Line 5
6  from dotenv import load_dotenv           # Line 6
7
8  # Load environment variables from .env file
9  load_dotenv()                            # Line 9
10
11
12 def get_env_var(key: str, default: str | None = None,
13                 required: bool = False) → str:
14     """Get an environment variable with optional default
15     and requirement check."""
16     value = os.getenv(key, default)        # Line 15
17     if required and not value:             # Line 16
18         raise ValueError(                 # Line 17
19             f"Missing required environment variable: {key}\n"
20             f"Please set it in your .env file. "
21             f"See .env.example for reference."
22         )
23     return value or ""                    # Line 21
24
25
26 # -- API Keys -----
27 GROQ_API_KEY = get_env_var("GROQ_API_KEY", required=True)
28 TAVILY_API_KEY = get_env_var("TAVILY_API_KEY", required=True)
29
30 # -- Model Configuration -----
31 MODEL_NAME = get_env_var("MODEL_NAME",
32                           default="llama-3.3-70b-versatile")
33 MAX_SEARCH_RESULTS = int(get_env_var("MAX_SEARCH_RESULTS",
34                                       default="5"))
35 MAX_REVISIONS = int(get_env_var("MAX_REVISIONS", default="2"))
36

```

```

37 # -- RAG Configuration -----
38 CHROMA_PERSIST_DIR = get_env_var("CHROMA_PERSIST_DIR",
39                                 default="./chroma_db")
40 EMBEDDING_MODEL = get_env_var("EMBEDDING_MODEL",
41                               default="sentence-transformers/all-MiniLM-L6-v2")
42 CHUNK_SIZE = int(get_env_var("CHUNK_SIZE", default="1000"))
43 CHUNK_OVERLAP = int(get_env_var("CHUNK_OVERLAP", default="200"))
44
45 # -- Set keys in environment (required by LangChain) ---
46 os.environ["GROQ_API_KEY"] = GROQ_API_KEY
47 os.environ["TAVILY_API_KEY"] = TAVILY_API_KEY

```

 [config/settings.py — Full Code](#)

🔗 Code Samjho Line-by-Line

Line-by-Line Breakdown:

Line 5: `import os` — Python ka built-in module. System se baat karne ke liye — files read karo, environment variables lo, paths banao.

Line 6: `from dotenv import load_dotenv` — `python-dotenv` library se aata hai. Ye `.env` file ko padhkar environment variables set karta hai. Matlab tumhara API key code mein hardcode nahi hoga (security best practice!).

Line 9: `load_dotenv()` — Ye function call karte hi, ye `.env` file dhundhta hai project root mein, usko padhta hai, aur har line ko `os.environ` mein daal deta hai. Example:

`.env` file mein: `GROQ_API_KEY=gsk_abc123xyz`

Ab Python mein: `os.getenv("GROQ_API_KEY") => "gsk_abc123xyz"`

Lines 12-21: `get_env_var()` function — Ye ek helper function hai jo environment variable padhta hai with error handling:

- **key:** `str` — Variable ka naam, e.g., `"GROQ_API_KEY"`
- **default:** `str | None = None` — Agar variable nahi mila toh ye value use karo
- **required:** `bool = False` — Agar `True` hai aur variable nahi mila, toh `ValueError` raise karo (app crash with clear message)
- **`str | None`** — Ye Python 3.10+ ka "union type" hai. Pehle `Optional[str]` likhte the.

Line 24-25: API Keys — Ye `required=True` hain. Matlab agar `.env` mein ye keys nahi hain, toh app start hi nahi hogi. Ye deliberate hai — bina API keys ke agents kuch kar hi nahi sakte.

Lines 28-31: Model Config — `MODEL_NAME` default `"llama-3.3-70b-versatile"` hai. Isko `.env` se override kar sakte ho. `MAX_SEARCH_RESULTS` aur `MAX_REVISIONS` ko `int()` mein wrap kiya kyunki env vars hamesha string hote hain.

Lines 33-37: RAG Config — ChromaDB kahan save hoga, kaunsa embedding model use hoga, chunks kitne bade honge — ye sab configurable hai.

Lines 39-40: Set in `os.environ` — LangChain integrations apne se `os.environ` se keys read karti hain. Toh hum explicitly set karte hain taaki koi issue na aaye.

💡 Yaad Rakhne Ka Tip

Syntax Yaad Karo:

```

1 # Environment variable padhne ka pattern:

```

```
2 import os
3 from dotenv import load_dotenv
4 load_dotenv()
5 api_key = os.getenv("MY_KEY", "default_value")
6
7 # Type hint with union (Python 3.10+):
8 def func(x: str | None = None) → str:
9     return x or "fallback"
```



Why .env Files?

Security: API keys ko code mein likhna bohot bura practice hai. Agar tum code ko GitHub pe push karte ho, toh koi bhi tumhare keys chura sakta hai. `.env` file ko `.gitignore` mein daalo taki ye kabhi Git repo mein na jaaye.

Flexibility: Development aur production mein different keys chahiye? Bas alag `.env` files use karo. Code change ki zaroorat nahi.

Team Work: `.env.example` file share karo (bina actual keys ke). Naye developer ko pata chal jayega ki kaunsi keys chahiye.

3 State Schema — state/schema.py

Kahani Se Samjho

Socho 4 doctors ek patient ka ilaaj kar rahe hain. Unhe ek **shared medical file** chahiye jismein saari information ho — symptoms, test results, diagnosis, treatment plan. Agar har doctor apni alag file rakhega, toh confusion hoga.

ResearchState exactly wahi **shared file** hai — ye ek TypedDict hai jo define karta hai ki pipeline mein kya-kya data flow karega. Har agent isko padh sakta hai aur update kar sakta hai.

3.1 Complete Code with Explanation

```
1 """
2 Shared state schema for the multi-agent research pipeline.
3
4 Uses TypedDict with Annotated reducer fields so that multiple
5 agents can safely append to list fields without overwriting
6 each other.
7 """
8
9 from __future__ import annotations      # Line 8
10 import operator                         # Line 10
11 from typing import Annotated, TypedDict # Line 11
12
13
14 class ResearchState(TypedDict):
15     """Central state shared across all agents."""
16
17     # -- User Input ----
18     topic: str
19     """The user's research query / topic."""
20
21     # -- Researcher Output ----
22     research_data: Annotated[list[str], operator.add]
23     """Raw search results collected by the Researcher agent.
24     Uses operator.add reducer -- each agent call *appends*."""
25
26     # -- Analyst Output ----
27     analysis: str
28     """Structured analysis with key themes, patterns."""
29
30     # -- Fact-Checker Output ----
31     fact_check_result: str
32     """Detailed fact-check report."""
33
34     fact_check_passed: bool
```

```

35     """Gate flag -- True = proceed, False = loop back."""
36
37     revision_count: int
38     """Number of research→fact-check loops. Capped."""
39
40     # -- Writer Output ----
41     final_report: str
42     """The polished, cited markdown report."""
43
44     # -- RAG Context ----
45     rag_context: Annotated[list[str], operator.add]
46     """Context from uploaded documents via ChromaDB RAG."""
47
48     # -- Pipeline Metadata ----
49     messages: Annotated[list[str], operator.add]
50     """Running log of agent activity messages."""
51
52     current_agent: str
53     """Name of the currently active agent."""
54
55     error: str
56     """Error message if something goes wrong."""

```

 [state/schema.py — Full Code](#)

Code Samjho Line-by-Line

Line-by-Line Deep Dive:

Line 8: `from __future__ import annotations` — Ye ek “magic import” hai. Isse Python type hints ko “lazy” banata hai — matlab type hints runtime pe evaluate nahi hote, sirf static analysis ke liye hote hain. Iska fayda: tum `list[str]` jaise modern syntax use kar sakte ho Python 3.9 mein bhi.

Line 10: `import operator` — Python ka built-in module for operators as functions. `operator.add` matlab `+` operator as a function. Ye LangGraph ko batata hai “list fields ko merge (concatenate) karo, overwrite mat karo.”

Line 11: `from typing import Annotated, TypedDict` —

- **TypedDict:** Ye ek special dict hai jismein har key ka type defined hota hai. Normal dict mein kuch bhi daal sakte ho, but TypedDict mein sirf defined keys allowed hain.
- **Annotated:** Ye type hints mein extra metadata attach karne ka mechanism hai. Yahan hum “reducer function” attach kar rahe hain.

Line 14: `class ResearchState(TypedDict):` — Ye humara **state container** hai. LangGraph mein har node (agent) ko ye state milti hai as input aur ye state return karti hai as output.

Annotated Reducer — Core LangGraph Concept

Ye sabse important concept hai is project mein. Samjho carefully:

Problem: Agar Researcher agent `messages = ["msg1"]` return kare, aur Analyst agent `messages = ["msg2"]` return kare — toh kya hoga? Normal dict mein “msg1”

overwrite ho jayega!

Solution: Reducer Pattern

```

1 # BIN Reducer (default behavior):
2 messages = ["msg2"] # Analyst ne Researcher ka data uda diya!
3
4 # WITH Reducer (operator.add):
5 messages: Annotated[list[str], operator.add]
6 # Ab: messages = ["msg1"] + ["msg2"] = ["msg1", "msg2"]
7 # Dono preserve hote hain!

```

Kaise kaam karta hai internally?

Jab LangGraph ek node ka output process karta hai:

1. Check karta hai — kya is field pe `Annotated` with reducer hai?
2. Agar haan — toh `reducer(old_value, new_value)` call karta hai
3. `operator.add` for lists: `[old] + [new]` = concatenation
4. Agar nahi — toh simply overwrite karta hai (last write wins)

√ Maths Behind the Scene

State Evolution Formula:

Maan lo S_t state hai time t pe, aur agent A_i output deta hai Δ_i :

$$S_{t+1}[\text{topic}] = \Delta_i[\text{topic}] \quad (\text{overwrite} \text{ — no reducer}) \quad (1)$$

$$S_{t+1}[\text{messages}] = S_t[\text{messages}] \oplus \Delta_i[\text{messages}] \quad (\text{append} \text{ — operator.add}) \quad (2)$$

$$\text{where } \oplus \equiv \text{list concatenation (operator.add)} \quad (3)$$

Matlab `topic` hamesha latest value rakhega (overwrite), lekin `messages` sab accumulate hote rahenge (append).

📚 Memory Trick

Yaad Rakhne Ka Trick:

“Annotated = Append, Normal = New (overwrite)”

Jab bhi list field mein sabka data rakhna ho → `Annotated[list, operator.add]`

Jab sirf latest value chahiye → normal type hint (`str`, `bool`, etc.)

3.2 State Fields Ka Role Table

Field	Type	Reducer?	Purpose (Hinglish)
topic	str	No	User ne kya research karna hai
research_data	list[str]	operator.add	Web search ke results, har baar append
analysis	str	No	Analyst ka structured output
fact_check_result	str	No	Fact checker ka detailed report

fact_check_passed	bool	No	True = Writer pe jao, False = wapas Researcher
revision_count	int	No	Kitni baar loop hua (max 2)
final_report	str	No	Writer ka final markdown report
rag_context	list[str]	operator.add	ChromaDB se mile document chunks
messages	list[str]	operator.add	UI ke liye activity log
current_agent	str	No	Abhi kaunsa agent active hai
error	str	No	Koi error aaya toh yahan store

4 Researcher Agent — agents/researcher.py

Kahani Se Samjho

Researcher agent ek **investigative journalist** jaisa hai. Usko topic milta hai, woh:

1. Pehle **3 strategic sub-queries** generate karta hai (LLM se)
2. Phir har query ke liye **Tavily web search** karta hai
3. Saath mein **ChromaDB mein uploaded documents** bhi search karta hai (RAG)
4. Aur **past research memory** bhi check karta hai
5. Sab results ko aggregate karke return karta hai

Agar ye revision round hai (Fact Checker ne wapas bheja), toh queries fact-checker feedback pe based hoti hain.

4.1 Complete Code with Detailed Commentary

```

1  """
2  Researcher Agent
3  -----
4  Uses Tavily web search AND ChromaDB RAG to gather data.
5  Strategy: generates 3 strategic sub-queries from the main
6  topic, then searches BOTH web and uploaded documents.
7  """
8
9  from langchain_groq import ChatGroq           # Line 10
10 from langchain_tavily import TavilySearch      # Line 11
11 from langchain_core.messages import (         # Line 12
12     SystemMessage, HumanMessage
13 )
14
15 from config.settings import MODEL_NAME, MAX_SEARCH_RESULTS
16 from state.schema import ResearchState        # Line 16
17
18 # -- Tool setup -----
19 search_tool = TavilySearch(                   # Line 19
20     max_results=MAX_SEARCH_RESULTS,           # Line 20
21     search_depth="advanced",                  # Line 21
22 )
23
24 llm = ChatGroq(model=MODEL_NAME, temperature=0) # Line 24
25
26
27
28 def researcher_node(state: ResearchState) → dict: # Line 27
29     """
30     Researcher agent node for the LangGraph pipeline.
31     """

```

```

32     topic = state["topic"] # Line 31
33     revision_count = state.get("revision_count", 0) # Line 32
34
35     # If revision loop, use fact-checker feedback
36     if revision_count > 0 and state.get("fact_check_result"):
37         query_prompt = f"""You are a research strategist.
38 A fact-checker has flagged issues with previous research
39 on: "{topic}"
40
41 Fact-checker feedback:
42 {state['fact_check_result']}
43
44 Generate exactly 3 targeted search queries to fill the
45 gaps and verify the flagged claims. Return ONLY the 3
46 queries, one per line, no numbering or bullets."""
47     else:
48         query_prompt = f"""You are a research strategist.
49 Generate exactly 3 focused, diverse search queries to
50 comprehensively research this topic: "{topic}"
51
52 Cover different angles: factual data, expert opinions,
53 and recent developments.
54 Return ONLY the 3 queries, one per line."""
55
56     # Generate sub-queries via LLM
57     query_response = llm.invoke([ # Line 58
58         SystemMessage(content="You are a research query "
59                             "generator. Output only the queries."),
60         HumanMessage(content=query_prompt),
61     ])
62
63     sub_queries = [q.strip() for q in # Line 63
64                   query_response.content.strip().split("\n")
65                   if q.strip()]
66     sub_queries = sub_queries[:3] # Safety cap # Line 65
67
68     # Fallback if LLM fails
69     if not sub_queries: # Line 68
70         sub_queries = [topic]
71
72     # -- Web Search (Tavily) -----
73     all_results = []
74     for query in sub_queries: # Line 72
75         try:
76             results = search_tool.invoke( # Line 74
77                 {"query": query}
78             )
79             if isinstance(results, list): # Line 76
80                 for r in results:
81                     if isinstance(r, dict):
82                         source = r.get("url",
83                                     "Unknown source")

```

```

84         content = r.get("content", "")
85         all_results.append(
86             f"[Source: {source}]\n{content}"
87         )
88     else:
89         all_results.append(str(r))
90     elif isinstance(results, str):
91         all_results.append(results)
92 except Exception as e:
93     all_results.append(
94         f"[Search error for '{query}']: {str(e)}"
95     )
96
97 # -- RAG Retrieval (ChromaDB) -----
98 rag_results = []
99 try:
100     from rag.vector_store import retrieve_context
101
102     # Search uploaded documents
103     doc_context = retrieve_context(                                # Line 94
104         topic,
105         collection_name="research_docs",
106         k=5
107     )
108     if doc_context:
109         rag_results.extend(doc_context)
110
111     # Search past research memory
112     past_context = retrieve_context(                                # Line 99
113         topic,
114         collection_name="past_research",
115         k=3
116     )
117     if past_context:
118         rag_results.extend(past_context)
119
120 except Exception as e:
121     rag_results.append(
122         f"[RAG retrieval note]: {str(e)}"
123     )
124
125 # -- Status Message -----
126 rag_count = len([r for r in rag_results
127     if not r.startswith("[RAG retrieval note]")])
128 status = (
129     f"Researcher: Searched {len(sub_queries)} "
130     f"web queries, found {len(all_results)} results"
131     + (f" + {rag_count} RAG chunks"
132        if rag_count > 0 else "")
133     + (" (revision round)"
134        if revision_count > 0 else "")
135 )

```

```

136
137     return {                                     # Line 115
138         "research_data": all_results,
139         "rag_context": rag_results,
140         "messages": [status],
141         "current_agent": "researcher",
142     }

```

 [agents/researcher.py — Full Code \(121 lines\)](#)

Code Samjho Line-by-Line

Critical Lines Explained:

Line 19-22: TavilySearch Setup

- `max_results=MAX_SEARCH_RESULTS` — Har query ke liye kitne results chahiye (default 5)
- `search_depth="advanced"` — Tavily ka advanced mode. Ye zyada thorough search karta hai — pages ke andar bhi content extract karta hai, sirf snippets nahi deta. Basic mode sirf surface-level results deta hai.

Line 24: LLM Setup — `temperature=0` matlab **deterministic output**. Researcher ko creative nahi hona hai — usko same input pe same queries generate karni hain. Temperature 0 se 1 tak hota hai:

- 0 = Same output every time (best for factual tasks)
- 0.3 = Thoda variation (Writer agent mein use hota hai)
- 1.0 = Maximum creativity (poetry, stories ke liye)

Lines 34-55: Query Generation Strategy — Ye sabse clever part hai. Instead of directly searching the topic, hum LLM se 3 diverse sub-queries generate karte hain. Kyun?

- Ek hi query se limited perspective milti hai
- 3 queries se “factual data”, “expert opinions”, “recent developments” — teen alag angles cover hote hain
- Revision round mein: Fact Checker ka feedback use karke **specific gaps** fill karte hain

Line 63-65: Query Parsing — LLM ka response ek multiline string hota hai. Hum:

1. `split("\n")` — Newline pe split
2. List comprehension se empty lines filter
3. `[:3]` se max 3 queries rakhte hain (safety — kabhi LLM 5 de deta hai)

Lines 72-86: Web Search Loop — Har sub-query ke liye Tavily call karte hain. Results mixed format mein aa sakte hain — list of dicts, list of strings, ya plain string. Isliye `isinstance()` checks hain. Har result ke saath source URL attach karte hain for citation.

Lines 88-104: RAG Retrieval — **Ye woh part hai jo is project ko special banata hai.** Web search ke saath-saath, hum:

1. `research_docs` collection mein uploaded PDFs search karte hain (k=5 results)
2. `past_research` collection mein pichli research sessions search karte hain (k=3 results)

Matlab agar user ne pehle “AI in Healthcare” pe research kiya tha, aur ab “AI in Diagnostics” pe kar raha hai — toh pichli research ka context bhi automatically mil jayega!

Line 115-120: Return Dict — Ye dict state mein merge hoga:

- `research_data` — Web results (`operator.add` se append honge)
- `rag_context` — Document chunks (`operator.add` se append honge)
- `messages` — Status log for UI
- `current_agent` — UI mein agent status update karne ke liye

Dhyan Do

Common Mistakes:

1. `search_depth="basic"` rakha toh Researcher shallow results dega — always “advanced” use karo research ke liye.
2. `temperature=0.7` rakh diya toh queries random hongi — har run mein alag results milenge.
3. `from rag.vector_store import retrieve_context` function ke andar hai (lazy import). Kyun? Kyunki agar ChromaDB install nahi hai ya crash karta hai, toh web search phir bhi kaam karega. Ye **graceful degradation** hai.

5 Analyst Agent — agents/analyst.py

Kahani Se Samjho

Analyst agent ek **detective** jaisa hai. Researcher ne saare clues (raw data) collect kar liye. Ab Analyst ko:

1. Clues mein se **patterns** dhundhne hain (“3 studies AI ke favour mein hain”)
2. **Contradictions** nikalne hain (“Study A kehta hai X, Study B kehta hai Y”)
3. **Consensus points** identify karne hain (sab agree karte hain ki...)
4. **Gaps** flag karne hain (“Kisi ne long-term effects nahi study kiye”)

Think of it as: Raw ingredients → Cooked analysis.

5.1 Complete Code with Commentary

```

1  """
2  Analyst Agent
3  -----
4  Takes raw research data and extracts structured insights:
5  key themes, patterns, contradictions, and consensus.
6  """
7
8  from langchain_groq import ChatGroq
9  from langchain_core.messages import (
10     SystemMessage, HumanMessage
11 )
12 from config.settings import MODEL_NAME
13 from state.schema import ResearchState
14
15 llm = ChatGroq(model=MODEL_NAME, temperature=0.1) # Line 15
16
17 ANALYST_SYSTEM_PROMPT = """You are an expert research
18 analyst. Your job is to synthesize raw research data
19 into a clear, structured analysis.
20
21 You MUST:
22 1. Identify 3-5 KEY THEMES or findings
23 2. Note any CONTRADICTIONS between sources
24 3. Highlight CONSENSUS points (claims supported
25    by multiple sources)
26 4. Flag any GAPS needing more investigation
27 5. Provide a brief SIGNIFICANCE assessment
28
29 Format output as structured analysis with markdown.
30 Be specific -- cite source URLs when referencing claims.
31 Do NOT make up information.""" # Line 29
32
33

```

```

34 def analyst_node(state: ResearchState) → dict:      # Line 32
35     """Analyst agent node for the LangGraph pipeline."""
36     topic = state["topic"]
37     research_data = state.get("research_data", [])
38     rag_context = state.get("rag_context", [])
39
40     if not research_data and not rag_context:        # Line 37
41         return {
42             "analysis": "No research data to analyze.",
43             "messages": ["Analyst: No data -- skipping."],
44             "current_agent": "analyst",
45         }
46
47     # Compile all research into one text block
48     compiled_data = "\n\n--\n\n".join(research_data)
49
50     # Add RAG context if available
51     rag_section = ""
52     if rag_context:                                  # Line 48
53         compiled_rag = "\n\n--\n\n".join(rag_context)
54         rag_section = f"""
55
56 DOCUMENT CONTEXT (from uploaded files & past research):
57 {compiled_rag}"""
58
59     analysis_prompt = f"""Analyze the following research
60 data on the topic: "{topic}"
61
62 WEB RESEARCH DATA:
63 {compiled_data}
64 {rag_section}
65
66 Provide a comprehensive structured analysis.
67 When citing, distinguish web and document sources."""
68
69     response = llm.invoke([                          # Line 64
70         SystemMessage(content=ANALYST_SYSTEM_PROMPT),
71         HumanMessage(content=analysis_prompt),
72     ])
73
74     analysis = response.content                        # Line 68
75
76     rag_note = (f" + {len(rag_context)} doc chunks"
77                 if rag_context else "")
78     return {
79         "analysis": analysis,
80         "messages": [f"Analyst: Analyzed "
81                     f"{len(research_data)} web results"
82                     f"{rag_note}"],
83         "current_agent": "analyst",
84     }

```

🔗 Code Samjho Line-by-Line

Key Design Decisions:

Line 15: `temperature=0.1` — Analyst ko thoda creative hona chahiye (patterns find karna creative task hai) lekin zyada nahi (data ke bahar nahi jaana chahiye). 0.1 ek “controlled creativity” hai.

System Prompt Analysis: System prompt mein 5 specific tasks defined hain. Ye **structured prompting** hai — LLM ko exact format batao toh output consistent aata hai. Bina structure ke LLM har baar alag format mein dega.

Lines 37-42: Empty Data Check — Agar Researcher ne kuch nahi diya (network error, etc.), toh Analyst gracefully handle karta hai instead of crashing. Ye **defensive programming** hai.

Lines 44-52: Data Compilation — Saare research results ko “--” separator se join karte hain. Kyun “—”? Kyunki ye markdown horizontal rule hai — LLM ko clearly pata chal jata hai ki ek source khatam hua, doosra shuru hua.

Lines 48-52: RAG Integration — Agar RAG context available hai, toh usko alag section mein add karte hain with label “DOCUMENT CONTEXT”. Ye LLM ko batata hai “ye web se nahi, uploaded files se aaya hai.”

Line 68: `response.content` — LangChain mein LLM response ek `AIMessage` object hota hai. `.content` se string text milta hai.

👤 Q: System Prompt vs User Prompt mein kya fark hai?

A: System prompt LLM ko batata hai “tu kaun hai, tera role kya hai” — ye ek baar set hota hai aur conversation ke dauran constant rehta hai. User prompt actual task hai — “ye data analyze karo.”

Example Analogy: System prompt = Job description (“Tum ek analyst ho”), User prompt = Daily task (“Aaj ye report analyze karo”).

Code mein:

```
1 response = llm.invoke([
2     SystemMessage(content="Tu analyst hai"), # Role
3     HumanMessage(content="Ye data analyze karo"), # Task
4 ])
```

6 Fact Checker Agent — agents/fact_checker.p

Kahani Se Samjho

Fact Checker ek **newspaper editor** jaisa hai jo publish hone se pehle har claim check karta hai:

- “Kya ye claim source data mein hai?” → VERIFIED
- “Partially support hai” → PARTIALLY VERIFIED
- “Koi evidence nahi mila” → UNVERIFIED

Agar bahut saare claims unverified hain, toh **NEEDS_REVISION** verdict deta hai aur Researcher ko wapas bhejta hai. Ye **feedback loop** is system ka sabse powerful feature hai — ye ensure karta hai ki output quality high ho.

6.1 Complete Code

```

1  """
2  Fact-Checker Agent
3  -----
4  Cross-references claims against raw research data.
5  Controls the feedback loop: if claims can't be verified,
6  routes back to the Researcher (up to MAX_REVISIONS).
7  """
8
9  from langchain_groq import ChatGroq
10 from langchain_tavily import TavilySearch
11 from langchain_core.messages import (
12     SystemMessage, HumanMessage
13 )
14 from config.settings import (
15     MODEL_NAME, MAX_REVISIONS, MAX_SEARCH_RESULTS
16 )
17 from state.schema import ResearchState
18
19 llm = ChatGroq(model=MODEL_NAME, temperature=0)    # Line 18
20
21 search_tool = TavilySearch(                        # Line 20
22     max_results=MAX_SEARCH_RESULTS,
23     search_depth="basic",
24 )
25
26 FACT_CHECKER_SYSTEM_PROMPT = """You are a meticulous
27 fact-checker. Your job is to verify claims made in a
28 research analysis against the provided source data.
29
30 For each major claim, you must:
31 1. CHECK if the claim is supported by source data
32 2. MARK each claim as: VERIFIED, PARTIALLY VERIFIED,
```

```
33     or UNVERIFIED
34 3. NOTE the source(s) that support or contradict
35
36 After reviewing all claims, provide:
37 - A SUMMARY of verification results
38 - A VERDICT: either "PASSED" or "NEEDS_REVISION"
39 - If NEEDS_REVISION: list specific questions needing
40   more research
41
42 You must output your verdict on the LAST LINE:
43 VERDICT: PASSED
44 or
45 VERDICT: NEEDS_REVISION
46
47 Be strict but fair. Minor gaps are okay.""" # Line 42
48
49
50 def fact_checker_node(state: ResearchState) → dict:
51     """Fact-checker agent node."""
52     topic = state["topic"]
53     analysis = state.get("analysis", "")
54     research_data = state.get("research_data", [])
55     revision_count = state.get("revision_count", 0)
56
57     if not analysis: # Line 51
58         return {
59             "fact_check_result": "No analysis to check.",
60             "fact_check_passed": True,
61             "messages": ["Fact-Checker: No analysis."],
62             "current_agent": "fact_checker",
63         }
64
65     compiled_sources = "\n\n---\n\n".join(research_data)
66
67     prompt = f"""Fact-check the following analysis
68 on "{topic}":
69
70 ANALYSIS TO VERIFY:
71 {analysis}
72
73 SOURCE DATA:
74 {compiled_sources}
75
76 Verify each major claim against the sources.
77 Provide detailed report and end with VERDICT."""
78
79     response = llm.invoke([ # Line 72
80         SystemMessage(content=FACT_CHECKER_SYSTEM_PROMPT),
81         HumanMessage(content=prompt),
82     ])
83
84     result = response.content # Line 76
```

```

85
86 # Parse verdict from response
87 passed = True # Line 78
88 if "VERDICT: NEEDS_REVISION" in result.upper(): # Line 79
89     if revision_count < MAX_REVISIONS: # Line 80
90         passed = False # Line 81
91
92 status_icon = "PASS" if passed else "REVISE"
93 status = (
94     f"Fact-Checker: "
95     + ("All claims verified -- proceeding to Writer"
96        if passed
97        else f"Flagged issues -- revision round "
98            f"{revision_count + 1}/{MAX_REVISIONS}")
99 )
100
101 return { # Line 92
102     "fact_check_result": result,
103     "fact_check_passed": passed,
104     "revision_count": revision_count
105                     + (0 if passed else 1),
106     "messages": [status],
107     "current_agent": "fact_checker",
108 }

```

 [agents/fact_checker.py](#) — Full Code (107 lines)

</> Code Samjho Line-by-Line

Critical Logic Analysis:

Line 18: `temperature=0` — Fact checking mein creativity ZERO chahiye. Ye sirf facts match karta hai — “haan ya nahi.”

Line 20-22: Search Tool — Fact Checker ke paas bhi Tavily hai, lekin `search_depth="basic"`. Kyun? Kyunki Fact Checker ko deep research nahi karni — sirf quick verification chahiye. “Basic” mode fast hai aur sufficient hai claims check karne ke liye.

Lines 78-81: Verdict Parsing — THE MOST CRITICAL LOGIC

```

1 passed = True # Default: pass
2 if "VERDICT: NEEDS_REVISION" in result.upper():
3     if revision_count < MAX_REVISIONS: # Only fail if
4         passed = False # revisions remain

```

Ye 4 lines pura feedback loop control karti hain:

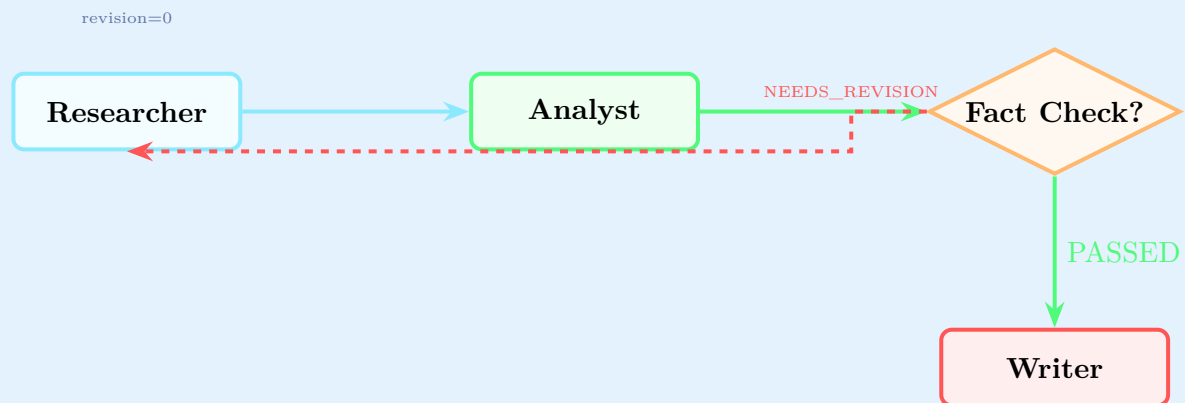
1. Default `passed = True` — agar verdict parse nahi ho paya, toh safe side pe pass
2. `result.upper()` — case-insensitive check (LLM kabhi “Needs_Revision” likh deta hai)
3. `revision_count < MAX_REVISIONS` — ye safety valve hai. Agar 2 baar revision ho chuka hai, toh chahe kitne bhi claims unverified hon, ab Writer ko bhejo. Bina iske infinite loop ban jayega!

Line 95: Revision Count Update

```
1 "revision_count": revision_count + (0 if passed else 1),
```

Agar pass hua toh count same, nahi toh +1. Ye ternary expression hai — compact way to conditionally increment.

Feedback Loop Ka Flow



Flow:

1. Round 1: Researcher searches → Analyst analyzes → Fact Checker says “NEEDS_REVISION” → revision_count becomes 1
2. Round 2: Researcher searches again (with feedback) → Analyst re-analyzes → Fact Checker says “NEEDS_REVISION” again → BUT revision_count(1) < MAX_REVISIONS(2) toh revision_count becomes 2
3. Round 3: Agar phir NEEDS_REVISION aaya — lekin ab revision_count(2) ≥ MAX_REVISIONS(2) toh passed = True force ho jata hai → Writer ko bhej do

Kyun max 2 revisions? Kyunki:

- Har revision mein 3 Tavily API calls + LLM calls hoti hain (cost + time)
- Infinite loop ka risk — agar LLM hamesha “NEEDS_REVISION” kehta rahe
- 2 rounds mein 90%+ claims usually verify ho jaate hain

7 Writer Agent — agents/writer.py

Kahani Se Samjho

Writer agent ek **professional report writer** hai. Uske paas aata hai:

- Verified analysis (Analyst se)
- Fact check notes (Fact Checker se)
- Raw research data (Researcher se)
- RAG document context (uploaded PDFs se)

Aur woh isse ek **polished markdown report** mein convert karta hai with:

- Executive Summary
- Key Findings (with inline citations [1], [2])
- Detailed Analysis
- Contradictions & Debates
- Conclusion
- Sources Bibliography

Plus — report complete hone ke baad, Writer session ko ChromaDB mein store karta hai for future memory!

7.1 Complete Code

```
1 """
2 Writer Agent
3 -----
4 Synthesizes verified analysis and research data into a
5 polished, well-structured markdown report with inline
6 citations, executive summary, and sources bibliography.
7 """
8
9 from langchain_groq import ChatGroq
10 from langchain_core.messages import (
11     SystemMessage, HumanMessage
12 )
13 from config.settings import MODEL_NAME
14 from state.schema import ResearchState
15
16 llm = ChatGroq(model=MODEL_NAME, temperature=0.3) # Line 15
17
18 WRITER_SYSTEM_PROMPT = """You are an expert research
19 report writer. You produce polished, professional
20 research reports in markdown format.
21
22 Your report MUST include these sections:
23 1. **Executive Summary** -- 2-3 paragraph overview
24 2. **Key Findings** -- Bulleted list with citations
25 3. **Detailed Analysis** -- Deep dive into themes
26 4. **Contradictions & Debates** -- Conflicting info
```

```

27 5. **Conclusion** -- Synthesis and open questions
28 6. **Sources** -- Numbered list of URLs
29
30 GUIDELINES:
31 - Use inline citations like [1], [2]
32 - Clear, professional tone for business audience
33 - Use markdown: headers, bullets, bold, blockquotes
34 - Include specific data points and statistics
35 - Report should be 800-1500 words
36 - Do NOT fabricate information""           # Line 34
37
38
39 def writer_node(state: ResearchState) → dict:   # Line 37
40     """Writer agent node."""
41     topic = state["topic"]
42     analysis = state.get("analysis", "")
43     fact_check_result = state.get("fact_check_result", "")
44     research_data = state.get("research_data", [])
45     rag_context = state.get("rag_context", [])
46
47     # Extract source URLs for bibliography
48     sources = []                               # Line 45
49     for item in research_data:
50         if "[Source:" in item:
51             try:
52                 url = item.split("[Source: ")[1] \
53                     .split("]")[0]
54                 if url not in sources \
55                     and url != "Unknown source":
56                     sources.append(url)
57             except (IndexError, ValueError):
58                 pass
59
60     # Extract RAG document sources
61     doc_sources = []                           # Line 57
62     for item in rag_context:
63         if "[RAG Source:" in item:
64             try:
65                 src = item.split("[RAG Source: ")[1] \
66                     .split("]")[0]
67                 if src not in doc_sources:
68                     doc_sources.append(src)
69             except (IndexError, ValueError):
70                 pass
71
72     # Build combined sources list
73     all_sources = []
74     for url in sources:
75         all_sources.append(url)
76     for src in doc_sources:
77         all_sources.append(f"(DOC) {src}")
78

```

```

79     sources_text = "\n".join(
80         f"[{i+1}] {src}"
81         for i, src in enumerate(all_sources)
82     )
83     compiled_data = "\n\n".join(
84         research_data[:10] # Cap to avoid token limits
85     )
86
87     # Add RAG context section
88     rag_section = ""
89     if rag_context:
90         compiled_rag = "\n\n".join(rag_context[:5])
91         rag_section = f"""
92
93 DOCUMENT CONTEXT (from uploaded files & past research):
94 {compiled_rag}"""
95
96     prompt = f"""Write a comprehensive research report
97 on: "{topic}"
98
99 STRUCTURED ANALYSIS:
100 {analysis}
101
102 FACT-CHECK NOTES:
103 {fact_check_result}
104
105 RAW RESEARCH DATA:
106 {compiled_data}
107 {rag_section}
108
109 AVAILABLE SOURCES FOR CITATION:
110 {sources_text}
111
112 Write the full report using source numbers [1], [2].
113 Include both web and document sources."""
114
115     response = llm.invoke([ # Line 104
116         SystemMessage(content=WRITER_SYSTEM_PROMPT),
117         HumanMessage(content=prompt),
118     ])
119
120     report = response.content # Line 108
121
122     # -- Store session in ChromaDB for memory ----
123     try: # Line 111
124         from rag.vector_store import (
125             store_research_session
126         )
127         chunks_stored = store_research_session(
128             topic, research_data, report
129         )
130         memory_msg = (f"Stored {chunks_stored} "

```

```

131         f"chunks to research memory")
132     except Exception:
133         memory_msg = "Memory storage skipped"
134
135     return {                                     # Line 120
136         "final_report": report,
137         "messages": [
138             "Writer: Final report completed",
139             memory_msg,
140         ],
141         "current_agent": "writer",
142     }

```

 agents/writer.py — Full Code (135 lines)

Code Samjho Line-by-Line

Key Insights:

Line 15: `temperature=0.3` — Writer ko “well-written” output chahiye — thodi creativity zaroori hai for good prose. But zyada creativity = hallucination risk. 0.3 is the sweet spot.

Lines 45-55: Source URL Extraction — Research data mein har result [Source: <https://...>] format mein hota hai. Writer ye URLs extract karke bibliography banata hai. String parsing use karta hai:

```

1  # Example data item:
2  "[Source: https://example.com/article]\nAI is growing"
3
4  # Parsing:
5  url = item.split("[Source: ")[1].split(" ")[0]
6  # Step 1: "[Source: ".split => "https://example.com/article]\n
   nAI..."
7  # Step 2: "]" .split => "https://example.com/article"

```

Lines 73-74: `research_data[:10]` — Ye token limit ke liye hai. LLM mein ek limit hoti hai kitna text ek baar mein process ho sakta hai (context window). Agar 50 results bhejenge toh overflow ho jayega. Top 10 sufficient hain.

Lines 111-118: Memory Storage — Ye is project ka “learning” feature hai. Har successful research session ka data ChromaDB mein store hota hai. Matlab:

- Aaj “AI Ethics” research kiya → stored in `past_research`
- Kal “AI Regulations” research karo → Researcher automatically “AI Ethics” ka context bhi dhundh lega
- System time ke saath **smarter** hota jaata hai!

Memory Trick

Temperature Cheat Sheet:

Agent	Temp	Reason
Researcher	0.0	Deterministic queries
Analyst	0.1	Controlled creativity
Fact Checker	0.0	Strict factual matching
Writer	0.3	Good prose quality

“Jitna factual kaam, utna kam temperature. Jitna creative kaam, utna zyada.”

8 LangGraph Workflow — graph/workflow.py

Kahani Se Samjho

Ab humne 4 agents bana liye. Lekin ye akele kaam nahi kar sakte — inhe ek **assembly line** mein lagana padega. LangGraph ye kaam karta hai — ye agents ko ek **directed graph** mein wire karta hai jahan:

- Nodes = Agents (functions)
- Edges = Data flow connections
- Conditional Edges = Decision points (“agar pass toh Writer, fail toh Researcher”)

Socho ek **factory assembly line** — car ka chassis pehle banta hai, phir engine lagta hai, phir quality check, phir painting. Agar quality check fail ho toh wapas engine section mein bhejo.

8.1 Complete Code

```

1  """
2  LangGraph Workflow
3  -----
4  Wires the 4 agents into a StateGraph pipeline with a
5  conditional feedback loop:
6      Researcher → Analyst → Fact-Checker → Writer
7  where Fact-Checker can route back to Researcher.
8  """
9
10 from langgraph.graph import StateGraph, END           # Line 10
11 from state.schema import ResearchState                 # Line 12
12 from agents.researcher import researcher_node         # Line 13
13 from agents.analyst import analyst_node              # Line 14
14 from agents.fact_checker import fact_checker_node    # Line 15
15 from agents.writer import writer_node                # Line 16
16
17
18 def should_continue(state: ResearchState) → str: # Line 19
19     """
20     Conditional edge after the Fact-Checker node.
21
22     Routes to:
23     - "writer"      → if fact-check passed
24     - "researcher" → if fact-check failed
25     """
26     if state.get("fact_check_passed", True):          # Line 27
27         return "writer"
28     else:
29         return "researcher"                           # Line 30
30

```

```

31
32 def build_graph() → StateGraph:                                # Line 33
33     """
34     Build and compile the research orchestrator graph.
35     Pipeline: researcher → analyst → fact_checker
36             → (writer | researcher)
37     """
38     # Create graph with shared state schema
39     workflow = StateGraph(ResearchState)                        # Line 40
40
41     # -- Register nodes -----
42     workflow.add_node("researcher", researcher_node)           # 43
43     workflow.add_node("analyst", analyst_node)                 # 44
44     workflow.add_node("fact_checker",
45                       fact_checker_node)                       # 45
46     workflow.add_node("writer", writer_node)                   # 46
47
48     # -- Define edges (pipeline flow) -----
49     workflow.set_entry_point("researcher")                      # Line 49
50
51     # Sequential edges
52     workflow.add_edge("researcher", "analyst")                 # Line 52
53     workflow.add_edge("analyst", "fact_checker")               # Line 53
54
55     # Conditional edge: fact_checker → writer OR
56     #                    fact_checker → researcher
57     workflow.add_conditional_edges(                             # Line 56
58         "fact_checker",                                         # Source node
59         should_continue,                                         # Decision function
60         {                                                         # Route map
61             "writer": "writer",
62             "researcher": "researcher",
63         },
64     )
65
66     # Terminal edge: writer → END
67     workflow.add_edge("writer", END)                            # Line 66
68
69     # -- Compile and return -----
70     graph = workflow.compile()                                   # Line 69
71     return graph
72
73
74 # Pre-built graph instance for import
75 research_graph = build_graph()                                # Line 74

```

 graph/workflow.py — Full Code (78 lines)

Code Samjho Line-by-Line

Every Line Matters:

Line 10: `from langgraph.graph import StateGraph, END`

- **StateGraph** — Ye class hai jo graph banati hai. Isko state schema pass karte ho (`ResearchState`), aur ye ensure karta hai ki har node correct types return kare.
- **END** — Ye ek special sentinel node hai. Jab koi node END se connect hota hai, matlab “pipeline yahan khatam.”

Line 19-30: `should_continue()` — The Decision Function

Ye function **conditional routing** implement karta hai. LangGraph is function ko Fact Checker ke baad call karta hai:

```
1 def should_continue(state: ResearchState) → str:
2     if state.get("fact_check_passed", True):
3         return "writer"          # Sab verified → Writer
4     else:
5         return "researcher"      # Issues found → Researcher
```

Note: `state.get("fact_check_passed", True)` — default True. Matlab agar key missing ho (edge case), toh assume “passed.” Ye **fail-safe** design hai.

Line 40: `StateGraph(ResearchState)` — Graph create karte waqt schema pass karte hain. Ye batata hai LangGraph ko ki “state ka structure ye hai, isko validate karo.”

Lines 43-46: Node Registration — `add_node(name, function)` se har agent register hota hai. Name string hona chahiye — ye name baad mein edges mein use hota hai.

Line 49: `set_entry_point("researcher")` — Pipeline kahan se start hogi. Hame-sha Researcher pehle.

Lines 52-53: Sequential Edges — `add_edge(A, B)` matlab “A ke baad B chalo.” Simple, unidirectional flow.

Lines 56-63: Conditional Edges — Ye sabse powerful feature hai:

```
1 workflow.add_conditional_edges(
2     "fact_checker",          # After this node...
3     should_continue,        # ...call this function...
4     {                        # ...and route based on return:
5         "writer": "writer",  # "writer" → writer node
6         "researcher": "researcher", # "researcher" → researcher
7     },
8 )
```

Route map ka format: `{function_return_value: node_name}`

Line 66: `add_edge("writer", END)` — Writer ke baad pipeline khatam.

Line 69: `workflow.compile()` — Graph ko “freeze” karta hai. Compile ke baad nodes/edges add nahi kar sakte. Ye compiled graph ab `.invoke()` ya `.stream()` se run ho sakta hai.

Line 74: Module-level instance — `research_graph = build_graph()` module load hote hi graph ban jaata hai. `from graph.workflow import research_graph` se koi bhi file isko use kar sakti hai.



LangGraph Internals — How the Engine Works

Jab tum `research_graph.invoke(initial_state)` call karte ho:

Step 1: Entry Point Check: LangGraph entry point dhundhta hai → “researcher”

Step 2: Node Execution: `researcher_node(state)` call hota hai. Ye ek dict return karta hai.

Step 3: State Merge: Return dict ko current state mein merge karta hai. `Annotated` fields ke liye reducer use hota hai (`operator.add`), baaki overwrite.

Step 4: Edge Resolution: “researcher” ke outgoing edges check karta hai → “analyst” milta hai

Step 5: Repeat: Analyst execute → merge → Fact Checker pe jaao

Step 6: Conditional Edge: Fact Checker ke baad `should_continue(state)` call hota hai

Step 7: Routing: Agar “writer” return hua → Writer execute karo. Agar “researcher” → wapas Step 2 pe jaao

Step 8: Terminal: Writer ke baad END milta hai → Final state return

recursion_limit: `{"recursion_limit": 25}` ka matlab hai max 25 node executions allowed hain. Agar feedback loop 25 se zyada baar chale (bug!), toh LangGraph exception throw karega.



Q: `.invoke()` vs `.stream()` mein kya fark hai?

A:

- `.invoke(state)` — Pura graph run karo aur final state return karo. Synchronous call. Simple use case ke liye.
- `.stream(state)` — Har node ke baad intermediate result yield karo. Generator return karta hai. UI ke liye perfect — real-time progress dikhao.

Code comparison:

```

1 # invoke -- sab ek baar mein
2 result = graph.invoke(initial_state)
3 print(result["final_report"])
4
5 # stream -- step by step
6 for event in graph.stream(initial_state):
7     for node_name, output in event.items():
8         print(f"{node_name} completed!")
9         # UI update karo

```

Humara `app.py` mein `.stream()` use hota hai taki Streamlit UI mein real-time agent progress dikhe.

9 RAG System — rag/vector_store.py

Kahani Se Samjho

RAG (Retrieval Augmented Generation) ko samajhne ke liye ek example lo:

Tum ek student ho jo exam ki tayyari kar raha hai. Tumhare paas do options hain:

1. **Bina notes:** Sab yaad se likho (LLM alone — hallucination risk)
2. **Notes ke saath:** Pehle relevant notes dhundho, phir unke basis pe likho (LLM + RAG — accurate!)

RAG exactly yahi karta hai:

1. **Retrieval** — Relevant documents dhundho (semantic search)
2. **Augmented** — Retrieved context ko LLM ke prompt mein add karo
3. **Generation** — LLM augmented context ke basis pe answer generate kare

Is project mein RAG do tarike se use hota hai:

- **Document RAG:** User PDF upload kare → chunks mein todo → ChromaDB mein store → research ke waqt search
- **Memory RAG:** Past research sessions store → future research mein context provide

9.1 Embedding Mathematics

Maths Behind the Scene

Embedding kya hai?

Text ko numbers (vectors) mein convert karna. Har word/sentence ko ek high-dimensional vector mein map karna.

Example:

$$\text{"AI is powerful"} \rightarrow [0.12, -0.34, 0.56, \dots, 0.78] \quad (384 \text{ dimensions}) \quad (4)$$

$$\text{"Machine learning rocks"} \rightarrow [0.11, -0.32, 0.55, \dots, 0.80] \quad (5)$$

$$\text{"I like pizza"} \rightarrow [0.89, 0.12, -0.67, \dots, -0.45] \quad (6)$$

Notice: "AI is powerful" aur "Machine learning rocks" ke vectors **similar** hain (close numbers), lekin "I like pizza" ka vector **bahut different** hai. Ye **semantic similarity** hai!

Cosine Similarity Formula:

$$\text{similarity}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \times \|\vec{b}\|} = \frac{\sum_{i=1}^n a_i \cdot b_i}{\sqrt{\sum_{i=1}^n a_i^2} \times \sqrt{\sum_{i=1}^n b_i^2}}$$

- Result range: $[-1, 1]$ (normalized vectors ke liye $[0, 1]$)
- 1.0 = identical meaning
- 0.0 = no relation
- Close to 1.0 = semantically similar

Humara model all-MiniLM-L6-v2:

- Output dimension: 384 (har text → 384-dimensional vector)
- Model size: ~80 MB (small, fast, free)
- Training: 1 billion+ sentence pairs pe trained
- Speed: ~14,000 sentences/second on CPU

9.2 Chunking Strategy

Kyun Chunking Zaroori Hai?

Ek 100-page PDF ko directly vector store mein nahi daal sakte. Kyun?

1. **Embedding models ka limit:** all-MiniLM-L6-v2 max 256 tokens process karta hai. Isse zyada text mein information loss hota hai.
2. **Retrieval precision:** Agar pura document ek vector hai, toh search “approximate” hogi. Chhote chunks se precise matching hoti hai.
3. **LLM context window:** Retrieved chunks LLM ke prompt mein jaate hain. Agar chunk bahut bada ho toh prompt overflow ho jayega.

Humara approach: RecursiveCharacterTextSplitter

```
1 text_splitter = RecursiveCharacterTextSplitter(
2     chunk_size=1000,          # Max 1000 characters per chunk
3     chunk_overlap=200,        # 200 chars overlap between chunks
4     separators=["\n\n", "\n", ". ", " ", ""],
5 )
```

Kaise kaam karta hai?

1. Pehle "\n\n" (double newline, i.e., paragraph break) pe split karne ki koshish
2. Agar chunk 1000 se bada hai, toh "\n" (single newline) pe try
3. Phir ". " (sentence boundary) pe try
4. Phir " " (word boundary) pe try
5. Last resort: character level split

Overlap kyun? Socho ek sentence do chunks ke beech split ho gaya — context kho jayega. 200 chars overlap se ek chunk ka end doosre chunk ke start mein include hota hai. Ye “sliding window” approach hai.

9.3 Complete Code

```
1 """
2 RAG Vector Store Manager
3 -----
4 Handles PDF ingestion, chunking, embedding, and retrieval
5 using ChromaDB (persistent) + HuggingFace embeddings.
6 """
7
8 import os
9 import hashlib
```

```

10 from typing import Optional
11
12 from langchain_huggingface import HuggingFaceEmbeddings
13 from langchain_chroma import Chroma
14 from langchain_community.document_loaders import PyPDFLoader
15 from langchain_text_splitters import (
16     RecursiveCharacterTextSplitter
17 )
18 from langchain_core.documents import Document
19
20 from config.settings import (
21     CHROMA_PERSIST_DIR, EMBEDDING_MODEL,
22     CHUNK_SIZE, CHUNK_OVERLAP,
23 )
24
25
26 # -- Embedding Model (singleton) -----
27 _embeddings = None # Line 28
28
29 def get_embeddings() → HuggingFaceEmbeddings: # Line 30
30     """Get or create the HuggingFace embedding model."""
31     global _embeddings
32     if _embeddings is None: # Line 33
33         _embeddings = HuggingFaceEmbeddings(
34             model_name=EMBEDDING_MODEL,
35             model_kwargs={"device": "cpu"},
36             encode_kwargs={"normalize_embeddings": True},
37         )
38     return _embeddings

```

 rag/vector_store.py — Key Functions

🔗 Code Samjho Line-by-Line

Singleton Pattern Explained:

`_embeddings = None` (module-level variable). Pehli baar `get_embeddings()` call hone pe model load hota hai (slow — ~2-3 seconds). Uske baad har call pe cached instance return hota hai (instant).

Kyun Singleton?

- Model load karna expensive hai (memory + time)
- Ek hi model multiple baar load karna wasteful hai
- global `_embeddings` se function ke bahar ki variable modify kar sakte hain

`normalize_embeddings=True` — Vectors ko unit length pe normalize karta hai. Iska fayda: cosine similarity calculate karna faster ho jata hai (denominator 1 ho jata hai toh sirf dot product chahiye).

```

1 def get_vectorstore(collection_name="research_docs"):
2     """Get or create a persistent ChromaDB store."""
3     os.makedirs(CHROMA_PERSIST_DIR, exist_ok=True)
4     return Chroma(
5         collection_name=collection_name,
6         embedding_function=get_embeddings(),

```

```

7     persist_directory=CHROMA_PERSIST_DIR,
8 )

```

Vector Store Access

Code Samjho Line-by-Line

os.makedirs() with **exist_ok=True** — Directory banao agar nahi hai. **exist_ok=True** se agar already hai toh error nahi aayega.

persist_directory — ChromaDB ka data disk pe save hota hai is directory mein. Matlab app restart hone pe bhi data rehta hai. Bina iske sab memory mein hota — app band karo, data gayab.

Two Collections:

1. "research_docs" — User uploaded PDFs ke chunks
2. "past_research" — Previous research sessions ka data

Ye ek database mein do tables jaisa hai — alag-alag data, same engine.

```

1 def ingest_documents(file_paths: list[str]) → dict:
2     """Load PDFs, chunk, embed, store in ChromaDB."""
3     text_splitter = RecursiveCharacterTextSplitter(
4         chunk_size=CHUNK_SIZE,
5         chunk_overlap=CHUNK_OVERLAP,
6         separators=["\n\n", "\n", ". ", " ", ""],
7     )
8
9     all_chunks = []
10    files_processed = []
11
12    for file_path in file_paths:
13        try:
14            loader = PyPDFLoader(file_path)
15            docs = loader.load() # List[Document]
16
17            # Add file metadata
18            file_name = os.path.basename(file_path)
19            for doc in docs:
20                doc.metadata["source_file"] = file_name
21
22            # Split into chunks
23            chunks = text_splitter.split_documents(docs)
24            all_chunks.extend(chunks)
25            files_processed.append(file_name)
26        except Exception as e:
27            files_processed.append(
28                f"{os.path.basename(file_path)} (ERROR: {e})"
29            )
30
31    if all_chunks:
32        # Generate unique IDs (content hash = no dupes)
33        ids = []
34        for chunk in all_chunks:
35            content_hash = hashlib.md5(

```

```

36         chunk.page_content.encode()
37     ).hexdigest()
38     ids.append(content_hash)
39
40     vectorstore = get_vectorstore("research_docs")
41     vectorstore.add_documents(
42         documents=all_chunks, ids=ids
43     )
44
45     return {
46         "chunks_added": len(all_chunks),
47         "files_processed": files_processed,
48     }

```

PDF Ingestion Function

MD5 Hash for Deduplication:

```

1 content_hash = hashlib.md5(
2     chunk.page_content.encode()
3 ).hexdigest()

```

Same content ka same hash hoga. Toh agar user same PDF dobara upload kare, toh duplicate chunks nahi banenge — ChromaDB same ID wale documents overwrite kar deta hai.

```

1 def retrieve_context(
2     query: str,
3     collection_name: str = "research_docs",
4     k: int = 5,
5 ) → list[str]:
6     """Retrieve most relevant chunks from ChromaDB."""
7     vectorstore = get_vectorstore(collection_name)
8
9     try:
10         results = vectorstore.similarity_search(
11             query, k=k
12         )
13     except Exception:
14         return []
15
16     if not results:
17         return []
18
19     formatted = []
20     for doc in results:
21         source = doc.metadata.get(
22             "source_file",
23             doc.metadata.get("source", "Unknown")
24         )
25         page = doc.metadata.get("page", "")
26         source_info = f"[RAG Source: {source}]"
27         if page != "":

```

```

28         source_info += f", Page {int(page) + 1}"
29         source_info += "]"
30         formatted.append(
31             f"{source_info}\n{doc.page_content}"
32         )
33
34     return formatted

```

Semantic Retrieval

Code Samjho Line-by-Line

Retrieval Process:

1. User query aati hai: "What are the effects of AI on healthcare?"
2. Query ko embed kiya jaata hai → 384-dimensional vector
3. ChromaDB mein stored vectors ke saath cosine similarity calculate hota hai
4. Top-k (default 5) most similar chunks return hote hain
5. Har chunk ke saath source metadata attach hota hai (filename, page number)

Note: page + 1 kyunki PyPDFLoader 0-indexed pages return karta hai, lekin humans 1-indexed count karte hain.

```

1  def store_research_session(
2      topic: str,
3      research_data: list[str],
4      report: str,
5  ) → int:
6      """Store completed session for future retrieval."""
7      text_splitter = RecursiveCharacterTextSplitter(
8          chunk_size=CHUNK_SIZE,
9          chunk_overlap=CHUNK_OVERLAP,
10     )
11
12     documents = []
13
14     # Store the final report
15     if report:
16         report_doc = Document(
17             page_content=report,
18             metadata={
19                 "source": f"Past Research: {topic}",
20                 "type": "report"
21             },
22         )
23         documents.append(report_doc)
24
25     # Store key research data (cap at 10)
26     for i, data in enumerate(research_data[:10]):
27         doc = Document(
28             page_content=data,
29             metadata={
30                 "source": f"Past Research: {topic}",
31                 "type": "research_data",

```

```

32         "index": i,
33     },
34 )
35 documents.append(doc)
36
37 if documents:
38     chunks = text_splitter.split_documents(documents)
39     ids = [
40         hashlib.md5(
41             f"{topic}_{i}_{c.page_content[:50]}"
42             .encode()
43         ).hexdigest()
44         for i, c in enumerate(chunks)
45     ]
46
47     vectorstore = get_vectorstore("past_research")
48     vectorstore.add_documents(
49         documents=chunks, ids=ids
50     )
51     return len(chunks)
52
53 return 0

```

Research Session Memory

Code Samjho Line-by-Line

Memory System Design:

Ye function Writer agent ke end mein call hota hai. Har successful research session ka:

1. **Final report** store hota hai (high-quality summarized content)
2. **Raw research data** ke top 10 results store hote hain (source material)

ID generation mein topic bhi include hai: `f"{topic}_{i}_{content[:50]}"`. Ye ensure karta hai ki same topic pe repeat research karne pe puraane chunks overwrite hon (same ID → overwrite).

Future Impact: Jab Researcher agent next time koi related topic search karega, `retrieve_context("past_research")` call se ye stored chunks automatic mil jayenge!

```

1 def get_collection_stats() → dict:
2     """Get document counts for all collections."""
3     stats = {}
4     for name in ["research_docs", "past_research"]:
5         try:
6             vs = get_vectorstore(name)
7             collection = vs._collection
8             stats[name] = collection.count()
9         except Exception:
10             stats[name] = 0
11     return stats
12
13

```

```
14 def clear_collection(  
15     collection_name: str = "research_docs"  
16 ) → bool:  
17     """Clear all documents from a collection."""  
18     try:  
19         vs = get_vectorstore(collection_name)  
20         collection = vs._collection  
21         all_ids = collection.get()["ids"]  
22         if all_ids:  
23             collection.delete(ids=all_ids)  
24         return True  
25     except Exception:  
26         return False
```

Utility Functions

Code Samjho Line-by-Line

vs._collection — Ye ChromaDB ka internal collection object access karta hai. Underscore prefix (_) ka matlab hai “private variable” — normally directly access nahi karna chahiye, lekin LangChain wrapper mein `.count()` method nahi hai, toh yahi option hai.

clear_collection — Pehle sabke IDs get karo, phir delete karo. ChromaDB mein bulk delete ke liye IDs list dena padta hai — “delete all” jaise shortcut nahi hai.

10 MCP Server — mcp_server.py

📖 Kahani Se Samjho

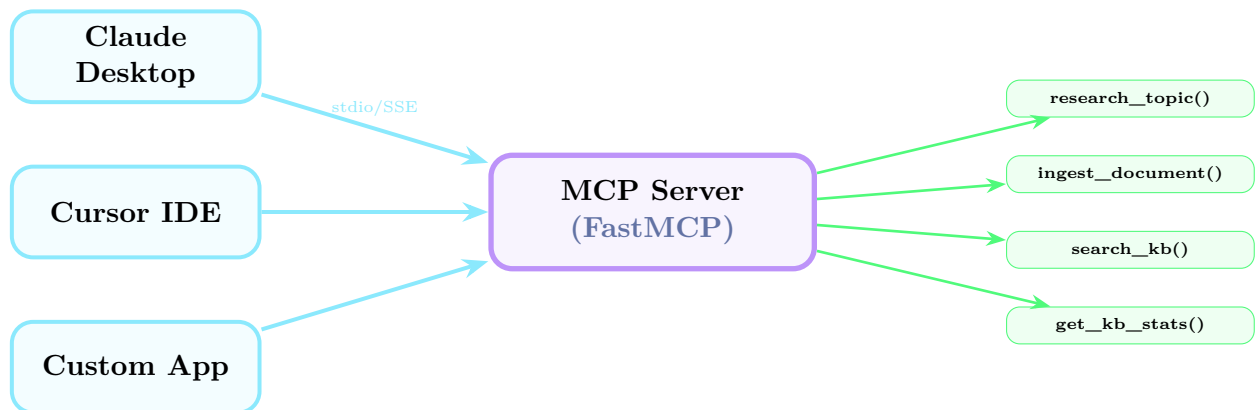
MCP (Model Context Protocol) ko samjhne ke liye socho:

Tumhare paas ek amazing research system hai (humara orchestrator). Ab tum chahte ho ki **Claude Desktop** ya **Cursor IDE** directly isse use kar sake — bina Streamlit UI open kiye.

MCP exactly ye karta hai — ye ek **standard protocol** hai (jaise HTTP web ke liye hai) jo AI tools ko ek dusre se baat karne deta hai. Humara MCP server 5 “tools” expose karta hai jo koi bhi MCP client call kar sakta hai.

Analogy: USB port socho. Koi bhi device (mouse, keyboard, pen drive) USB mein lag sakta hai kyunki standard hai. MCP wahi hai AI tools ke liye — ek universal plug.

10.1 MCP Architecture



10.2 Complete Code

```

1  """
2  MCP Server -- Research Orchestrator as a Tool
3  -----
4  Exposes the multi-agent research pipeline as MCP tools.
5  Any MCP-compatible client can call these tools.
6
7  Usage:
8      # Test with MCP Inspector
9      fastmcp dev mcp_server.py
10
11     # Run as stdio server
12     python mcp_server.py
  
```

```
13 """
14
15 import sys
16 import os
17
18 sys.path.insert(0, os.path.dirname(
19     os.path.abspath(__file__)
20 ))
21
22 from fastmcp import FastMCP # Line 18
23
24 # -- Initialize MCP Server -----
25 mcp = FastMCP( # Line 21
26     "ResearchOrchestrator",
27     description=(
28         "Multi-Agent Research Orchestrator -- "
29         "4 AI agents collaborate to research, "
30         "analyze, fact-check, and write reports."
31     ),
32 )
33
34
35 # -- Tool 1: Research a Topic -----
36 @mcp.tool() # Line 31
37 def research_topic(topic: str) → str:
38     """
39     Run the full multi-agent research pipeline.
40     4 agents collaborate to produce a comprehensive,
41     fact-checked, cited research report.
42
43     Args:
44         topic: The research topic or question.
45
46     Returns:
47         A polished markdown research report.
48     """
49     from graph.workflow import research_graph
50
51     initial_state = {
52         "topic": topic,
53         "research_data": [],
54         "analysis": "",
55         "fact_check_result": "",
56         "fact_check_passed": False,
57         "revision_count": 0,
58         "final_report": "",
59         "rag_context": [],
60         "messages": [],
61         "current_agent": "",
62         "error": "",
63     }
64
```

```

65     try:
66         result = research_graph.invoke(
67             initial_state, {"recursion_limit": 25}
68         )
69         report = result.get("final_report", "")
70         if report:
71             return report
72         elif result.get("error"):
73             return f"Pipeline error: {result['error']}"
74         else:
75             return "No report generated."
76     except Exception as e:
77         return f"Error: {str(e)}"
78
79
80 # -- Tool 2: Ingest Document -----
81 @mcp.tool()
82 def ingest_document(file_path: str) → str:
83     """Ingest a PDF into the RAG knowledge base."""
84     from rag.vector_store import ingest_documents
85
86     if not os.path.exists(file_path):
87         return f"Error: File not found at '{file_path}'"
88     if not file_path.lower().endswith(".pdf"):
89         return "Error: Only PDF files supported."
90
91     try:
92         result = ingest_documents([file_path])
93         return (
94             f"Ingested {result['chunks_added']} chunks"
95             f" from {'', '.join(result['files_processed'])}"
96         )
97     except Exception as e:
98         return f"Error: {str(e)}"
99
100
101 # -- Tool 3: Search Knowledge Base -----
102 @mcp.tool()
103 def search_knowledge_base(
104     query: str, num_results: int = 5
105 ) → str:
106     """Search the RAG knowledge base."""
107     from rag.vector_store import retrieve_context
108
109     try:
110         doc_results = retrieve_context(
111             query, collection_name="research_docs",
112             k=num_results
113         )
114         past_results = retrieve_context(
115             query, collection_name="past_research",
116             k=3

```

```

117         )
118         all_results = doc_results + past_results
119
120         if not all_results:
121             return "No relevant documents found."
122         return "\n\n---\n\n".join(all_results)
123     except Exception as e:
124         return f"Error: {str(e)}"
125
126
127 # -- Tool 4: KB Stats -----
128 @mcp.tool()
129 def get_kb_stats() → str:
130     """Get RAG knowledge base statistics."""
131     from rag.vector_store import get_collection_stats
132
133     try:
134         stats = get_collection_stats()
135         return (
136             f"Knowledge Base Stats:\n"
137             f"    Uploaded Docs: "
138             f"{stats.get('research_docs', 0)} chunks\n"
139             f"    Research Memory: "
140             f"{stats.get('past_research', 0)} chunks"
141         )
142     except Exception as e:
143         return f"Error: {str(e)}"
144
145
146 # -- Tool 5: Clear KB -----
147 @mcp.tool()
148 def clear_knowledge_base() → str:
149     """Clear all documents and memory."""
150     from rag.vector_store import clear_collection
151
152     try:
153         clear_collection("research_docs")
154         clear_collection("past_research")
155         return "Knowledge base cleared."
156     except Exception as e:
157         return f"Error: {str(e)}"
158
159
160 # -- Resource: Project Info -----
161 @mcp.resource("info://orchestrator") # Line 153
162 def get_info() → str:
163     """Information about the Research Orchestrator."""
164     return ""
165 Multi-Agent Research Orchestrator v1.0
166
167 Agents:
168     Researcher -- Web search + RAG retrieval

```

```

169 Analyst      -- Pattern extraction, gap analysis
170 Fact Checker -- Claim verification with feedback
171 Writer       -- Report synthesis with citations
172
173 Tech Stack:
174 LLM: Groq (llama-3.3-70b-versatile)
175 Search: Tavily (advanced depth)
176 Vector Store: ChromaDB (persistent)
177 Embeddings: HuggingFace all-MiniLM-L6-v2
178 Orchestration: LangGraph StateGraph
179 UI: Streamlit
180 Protocol: MCP (Model Context Protocol)
181 " " "
182
183
184 # -- Run Server -----
185 if __name__ == "__main__": # Line 175
186     mcp.run(transport="stdio")

```

 mcp_server.py — Full Code (208 lines)

🔗 Code Samjho Line-by-Line

MCP Concepts Explained:

@mcp.tool() Decorator: Ye kisi bhi function ko MCP tool bana deta hai. Jab Claude Desktop ya Cursor connect karta hai, toh ye tools automatically discover ho jaate hain. Function ka docstring tool ka description ban jaata hai, aur parameters automatically schema mein convert hote hain.

@mcp.resource("info://orchestrator"): Resources read-only data hain jo client access kar sakta hai. Tools ke opposite — tools actions perform karti hain, resources sirf information provide karti hain.

transport="stdio": MCP do transports support karta hai:

- **stdio** — Standard input/output streams se communicate. Claude Desktop aur Cursor ye use karte hain. Server ek subprocess ke roop mein run hota hai.
- **sse** — Server-Sent Events (HTTP-based). Web applications ke liye.

Claude Desktop mein setup:

```

# claude_desktop_config.json mein add karo:
{
  "mcpServers": {
    "research": {
      "command": "python",
      "args": ["C:/path/to/mcp_server.py"]
    }
  }
}

```

Ab Claude Desktop mein “Research AI in healthcare” type karo aur Claude automatically humara `research_topic()` tool call karega!

 Q: MCP vs REST API mein kya fark hai?

A:

	MCP	REST API
Purpose	AI tool interoperability	General web APIs
Discovery	Auto-discovery (tools, re-sources)	Manual (docs/Swagger)
Transport	stdio or SSE	HTTP
Schema	Function signatures auto-generate	Manually define endpoints
Clients	Claude, Cursor, etc.	Any HTTP client
State	Stateful connection	Stateless requests

MCP specifically AI-to-AI communication ke liye designed hai, jabki REST general purpose hai. MCP mein tool discovery automatic hai — client connect hota hai aur server batata hai “mere paas ye 5 tools hain.”

11 Streamlit UI — app.py Deep Dive

Kahani Se Samjho

app.py humari “showroom” hai — ye woh jagah hai jahan user interact karta hai. Is file mein 700+ lines hain covering:

1. Premium CSS with glassmorphism design
2. Session state management
3. Sidebar with real-time agent status
4. RAG document upload interface
5. Research pipeline execution with streaming
6. Results display with download option

11.1 Page Configuration

```
1 import sys, os, time
2 import streamlit as st
3
4 sys.path.insert(0,
5     os.path.dirname(os.path.abspath(__file__)))
6 )
7
8 # Page Config -- MUST be first Streamlit call!
9 st.set_page_config(
10     page_title="Multi-Agent Research Orchestrator",
11     page_icon="microscope",
12     layout="wide",           # Full width layout
13     initial_sidebar_state="expanded",
14 )
```

 app.py — Page Config

Dhyan Do

st.set_page_config() **MUST** be the first Streamlit command in your script. Agar pehle koi aur st. call aaye (even st.write()), toh error aayega: “set_page_config() can only be called once and must be the first Streamlit command.”

11.2 Premium CSS Design

 Glassmorphism CSS

```
1 /* Glass Card Effect */
2 .glass-card {
```

```

3   background: rgba(255, 255, 255, 0.03);
4   border: 1px solid rgba(255, 255, 255, 0.08);
5   border-radius: 16px;
6   padding: 1.5rem;
7   margin: 1rem 0;
8   backdrop-filter: blur(10px);
9   -webkit-backdrop-filter: blur(10px);
10  transition: all 0.3s ease;
11 }
12 .glass-card:hover {
13   border-color: rgba(102, 126, 234, 0.3);
14   box-shadow: 0 8px 32px rgba(102, 126, 234, 0.1);
15 }

```



Glassmorphism Kya Hai?

Glassmorphism ek modern UI design trend hai jismein elements **frosted glass** jaise dikhte hain:

- `rgba(255, 255, 255, 0.03)` — Almost transparent white background (3% opacity)
- `backdrop-filter: blur(10px)` — Background content ko blur karta hai glass ke peeche
- `border: 1px solid rgba(255, 255, 255, 0.08)` — Subtle edge glow

Ye dark theme ke saath combine karke “premium” feel deta hai — Apple, Microsoft ye design use karte hain.

11.3 Session State Management

```

1  # Session State -- Streamlit reruns script on every
2  # interaction, so state must be preserved externally
3  if "research_running" not in st.session_state:
4      st.session_state.research_running = False
5  if "research_complete" not in st.session_state:
6      st.session_state.research_complete = False
7  if "final_report" not in st.session_state:
8      st.session_state.final_report = ""
9  if "agent_messages" not in st.session_state:
10     st.session_state.agent_messages = []
11  if "agent_statuses" not in st.session_state:
12     st.session_state.agent_statuses = {
13         "researcher": "idle",
14         "analyst": "idle",
15         "fact_checker": "idle",
16         "writer": "idle",
17     }
18  if "error" not in st.session_state:
19     st.session_state.error = ""

```

 Session State Initialization



Streamlit Ka Execution Model

Streamlit ka sabse important concept samjho:

Har interaction pe pura script re-run hota hai!

Matlab jab user button click kare, slider move kare, ya text type kare — Streamlit pura app.py top se bottom tak dobara execute karta hai. Isliye:

- Variables normally reset ho jaate hain (har run pe naye bante hain)
- `st.session_state` ek persistent dictionary hai jo runs ke beech data preserve karti hai
- `if "key" not in st.session_state:` — sirf pehli baar initialize karo, baad mein skip

Analogy: Socho ek whiteboard hai. Har baar jab koi button dabaye, whiteboard saaf ho jaata hai aur dobara likho. `session_state` ek separate notebook hai jisko whiteboard saaf karne se kuch nahi hota.

11.4 Pipeline Execution with Streaming

```

1  if start_btn and topic:
2      # Reset state
3      st.session_state.research_running = True
4      st.session_state.research_complete = False
5      st.session_state.final_report = ""
6      st.session_state.agent_messages = []
7      st.session_state.error = ""
8      st.session_state.agent_statuses = {
9          "researcher": "idle", "analyst": "idle",
10         "fact_checker": "idle", "writer": "idle",
11     }
12
13     try:
14         from graph.workflow import research_graph
15     except ValueError as e:
16         st.error(f"Configuration Error: {e}")
17         st.stop() # Stop execution entirely
18
19     # Initial state for the graph
20     initial_state = {
21         "topic": topic,
22         "research_data": [],
23         "analysis": "",
24         "fact_check_result": "",
25         "fact_check_passed": False,
26         "revision_count": 0,
27         "final_report": "",
28         "rag_context": [],
29         "messages": [],
30         "current_agent": "",
31         "error": "",
32     }

```

```

33
34     # Stream the graph execution
35     status_container = st.empty()
36     progress_bar = st.progress(0)
37
38     try:
39         step_count = 0
40         agent_order = ["researcher", "analyst",
41                        "fact_checker", "writer"]
42
43         for event in research_graph.stream(
44             initial_state,
45             {"recursion_limit": 25}
46         ):
47             for node_name, node_output in event.items():
48                 step_count += 1
49
50                 # Update agent statuses for UI
51                 for agent in agent_order:
52                     if agent == node_name:
53                         st.session_state \
54                             .agent_statuses[agent] = "done"
55                     elif st.session_state \
56                         .agent_statuses[agent] != "done":
57                         next_idx = agent_order.index(agent)
58                         current_idx = agent_order.index(
59                             node_name
60                         ) if node_name in agent_order else -1
61                         if next_idx == current_idx + 1:
62                             st.session_state \
63                                 .agent_statuses[agent] \
64                                 = "active"
65
66                 # Collect messages
67                 if isinstance(node_output, dict):
68                     new_msgs = node_output.get(
69                         "messages", []
70                     )
71                     if new_msgs:
72                         st.session_state \
73                             .agent_messages \
74                             .extend(new_msgs)
75
76                 if node_output.get("final_report"):
77                     st.session_state.final_report \
78                         = node_output["final_report"]
79
80                 if node_output.get("error"):
81                     st.session_state.error \
82                         = node_output["error"]
83
84                 # Update progress bar

```

```

85         completed = sum(
86             1 for s in st.session_state
87             .agent_statuses.values()
88             if s == "done"
89         )
90         progress_bar.progress(completed / 4)
91
92         # Mark all as done
93         for agent in agent_order:
94             st.session_state.agent_statuses[agent] = "done"
95
96         st.session_state.research_complete = True
97         st.session_state.research_running = False
98         progress_bar.progress(1.0)
99
100     except Exception as e:
101         st.session_state.error = str(e)
102         st.session_state.research_running = False
103         st.error(f"Pipeline Error: {e}")
104
105     st.rerun() # Force re-render with new state

```

 app.py — Research Pipeline Execution

🔗 Code Samjho Line-by-Line

Streaming Logic Breakdown:

research_graph.stream() ek generator return karta hai. Har yield pe ek event aata hai jo dictionary hai: {node_name: node_output}.

Agent Status Update Logic:

1. Jab ek node complete ho: uska status “done” set karo
2. Next node in sequence: uska status “active” set karo (for animated UI)
3. Ye sidebar mein real-time pills update karti hain (idle → active → done)

st.rerun() — Pipeline complete hone ke baad, script ko forcefully re-run karo. Kyun? Kyunki streaming ke dauran UI elements update nahi hote (Streamlit ki limitation). Rerun se pura page redraw hota hai with final state.

st.empty() — Ek placeholder create karta hai jo baad mein replace ho sakta hai. Pipeline running ke dauran status message dikhata hai, complete hone pe empty kar deta hai.

st.progress(completed / 4) — 4 agents hain. Jitne done, utna progress bar fill. $0/4 = 0\%$, $4/4 = 100\%$.

11.5 RAG Document Upload UI

```

1 with st.expander("Upload Documents (RAG)", expanded=False):
2     uploaded_files = st.file_uploader(
3         "Upload PDF documents",
4         type=["pdf"],
5         accept_multiple_files=True,

```

```

6         label_visibility="collapsed",
7         key="pdf_uploader",
8     )
9
10    if uploaded_files:
11        if st.button("Ingest Documents",
12                    key="ingest_btn",
13                    use_container_width=True):
14            with st.spinner("Processing PDFs..."):
15                try:
16                    from rag.vector_store import (
17                        ingest_from_streamlit_uploads
18                    )
19                    result = ingest_from_streamlit_uploads(
20                        uploaded_files
21                    )
22                    st.success(
23                        f"Ingested {result['chunks_added']}"
24                        f" chunks from "
25                        f"{len(result['files_processed'])}"
26                        f" file(s)"
27                    )
28                except Exception as e:
29                    st.error(f"Ingestion failed: {e}")

```

 app.py — PDF Upload Section

🔗 Code Samjho Line-by-Line

UI Components Used:

- `st.expander()` — Collapsible section. Space-efficient design.
- `st.file_uploader()` — File upload widget. `type=["pdf"]` se sirf PDFs accept hote hain.
- `accept_multiple_files=True` — Multiple files ek baar mein upload
- `st.spinner()` — Loading indicator jab tak code execute ho raha hai
- `st.success()/st.error()` — Green/Red notification boxes

`ingest_from_streamlit_uploads()` — Special function jo Streamlit ke `UploadedFile` objects ko temporary files mein write karta hai (kyunki `PyPDFLoader` ko file path chahiye, in-memory buffer nahi), phir process karta hai, phir temp files delete karta hai.

12 Environment Setup — .env.example

.env.example — Template

```
1 # Required API Keys
2 GROQ_API_KEY=gsk_your_groq_api_key_here
3 TAVILY_API_KEY=tvly-your-tavily-api-key-here
4
5 # Optional: Override model
6 MODEL_NAME=llama-3.3-70b-versatile
7 MAX_SEARCH_RESULTS=5
8 MAX_REVISIONS=2
9
10 # RAG Configuration
11 CHROMA_PERSIST_DIR=./chroma_db
12 EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
13 CHUNK_SIZE=1000
14 CHUNK_OVERLAP=200
```

Yaad Rakhne Ka Tip

API Keys Kaise Milenge?

1. **Groq:** <https://console.groq.com> pe jaao → Sign up → API Keys section → “Create API Key” → Copy. Free tier mein per-minute rate limits hain.
2. **Tavily:** <https://tavily.com> pe jaao → Sign up → Dashboard → API Key copy. Free tier: 1000 searches/month.

12.1 Running the Project

Setup Commands

```
# 1. Clone / navigate to project
cd Multi-Agent-Research-Orchestrator

# 2. Create virtual environment
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # Mac/Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. Create .env file
copy .env.example .env
# Edit .env with your actual API keys

# 5. Run Streamlit UI
streamlit run app.py
```

```
# 6. (Optional) Run MCP Server
python mcp_server.py

# 7. (Optional) Test MCP with Inspector
fastmcp dev mcp_server.py
```

13 Advanced Concepts — Deep Technical Dive

13.1 LangGraph State Machine Theory

State Machine as Directed Graph

LangGraph internally ek **Finite State Machine (FSM)** implement karta hai:

$$\text{FSM} = (Q, \Sigma, \delta, q_0, F) \quad (7)$$

$$Q = \{\text{researcher, analyst, fact_checker, writer}\} \quad (\text{states}) \quad (8)$$

$$\Sigma = \{\text{state updates from each node}\} \quad (\text{input alphabet}) \quad (9)$$

$$\delta : Q \times \Sigma \rightarrow Q \quad (\text{transition function}) \quad (10)$$

$$q_0 = \text{researcher} \quad (\text{initial state}) \quad (11)$$

$$F = \{\text{END}\} \quad (\text{final/accepting state}) \quad (12)$$

Transition Function δ :

$$\delta(\text{researcher}, _) = \text{analyst} \quad (\text{always}) \quad (13)$$

$$\delta(\text{analyst}, _) = \text{fact_checker} \quad (\text{always}) \quad (14)$$

$$\delta(\text{fact_checker}, s) = \begin{cases} \text{writer} & \text{if } s.\text{fact_check_passed} = \text{True} \\ \text{researcher} & \text{if } s.\text{fact_check_passed} = \text{False} \end{cases} \quad (15)$$

$$\delta(\text{writer}, _) = \text{END} \quad (\text{always}) \quad (16)$$

Cycle Detection: $\text{fact_checker} \rightarrow \text{researcher} \rightarrow \text{analyst} \rightarrow \text{fact_checker}$ — ye cycle hai. **revision_count** se ye bounded cycle hai (max 2 iterations), isliye termination guaranteed hai.

13.2 Vector Database Internals

ChromaDB HNSW Index

ChromaDB internally **HNSW (Hierarchical Navigable Small World)** algorithm use karta hai for approximate nearest neighbor search.

Naive Approach: Har query ke liye saare vectors ke saath cosine similarity calculate karo $\rightarrow O(n)$ time. 1 million documents ke liye slow!

HNSW Approach:

1. Vectors ka ek multi-layer graph banao
2. Top layer: sparse connections (long-range jumps)

3. Bottom layer: dense connections (local precision)
4. Search: Top layer se start, gradually zoom in to bottom layer
5. Time complexity: $O(\log n)$ — exponentially faster!

Trade-off: Approximate hai — 100% perfect nearest neighbor nahi milega, lekin 95%+ accuracy with 100x speed improvement.

13.3 Token Economics

Cost Analysis Per Research Session

Component	API Calls	Tokens (est.)	Cost
Query Generation	1 LLM call	~500	Free (Groq)
Web Search	3 Tavily calls	N/A	Free tier
Analysis	1 LLM call	~3000	Free (Groq)
Fact Check	1 LLM call	~3000	Free (Groq)
Report Writing	1 LLM call	~4000	Free (Groq)
Embeddings	Local model	N/A	\$0 (CPU)
Per Session	~6 calls	~10,500	FREE
With 2 Revisions	~14 calls	~25,000	FREE

Key Insight: Pura system practically free hai! Groq free tier pe rate limits hain (30 requests/minute), but small projects ke liye sufficient. Tavily free tier: 1000 searches/month. Embeddings local hain (no API cost).

13.4 Error Handling Patterns

```

1 # Pattern 1: Graceful Degradation (Researcher)
2 try:
3     from rag.vector_store import retrieve_context
4     doc_context = retrieve_context(topic)
5 except Exception:
6     rag_results.append("[RAG retrieval note]: ...")
7 # RAG fails? Web search still works!
8
9 # Pattern 2: Fail-Safe Default (Fact Checker)
10 if state.get("fact_check_passed", True):
11     # Default True = if key missing, assume passed
12     return "writer"
13
14 # Pattern 3: Safety Cap (Researcher)
15 sub_queries = sub_queries[:3] # Never more than 3
16 # Even if LLM returns 10 queries, we cap at 3
17
18 # Pattern 4: Token Limit Protection (Writer)
19 compiled_data = "\n\n".join(research_data[:10])
20 # Only top 10 results to avoid context overflow

```

Error Handling Patterns Used

14 Interview Preparation — 50 Questions with In-Depth Answers

Kahani Se Samjho

Ye section tumhe interview-ready banayega. Har question ke saath detailed answer hai jo interviewer ko impress karega. Questions ko 5 categories mein divide kiya hai:

1. **Architecture & Design** (Q1-Q10)
2. **LangGraph & Orchestration** (Q11-Q20)
3. **RAG & Vector Databases** (Q21-Q30)
4. **MCP & Integration** (Q31-Q37)
5. **Production & Optimization** (Q38-Q50)

14.1 Architecture & Design (Q1–Q10)

Q1: Is project ka high-level architecture explain karo.

A: Ye ek **multi-agent research orchestration system** hai jo LangGraph State-Graph pattern follow karta hai.

4 Specialized Agents:

1. **Researcher:** Tavily web search + ChromaDB RAG se data collect karta hai. 3 strategic sub-queries generate karke diverse coverage ensure karta hai.
2. **Analyst:** Raw data ko structured analysis mein convert karta hai — themes, contradictions, consensus, gaps identify karta hai.
3. **Fact Checker:** Claims verify karta hai against source data. NEEDS_REVISION verdict se feedback loop trigger hota hai (max 2 revisions).
4. **Writer:** Verified analysis ko polished markdown report mein convert karta hai with inline citations. Saath mein session ko ChromaDB memory mein store karta hai.

Key Design Patterns:

- **Shared State (TypedDict with reducers):** Agents ek central state share karte hain. List fields pe `operator.add` reducer se data accumulate hota hai.
- **Conditional Routing:** Fact Checker ke output pe dynamic decision — Writer ya wapas Researcher.
- **Graceful Degradation:** RAG fail ho toh web search chalti rehti hai.

Q2: Kyun 4 alag agents use kiye? Ek hi LLM se sab nahi ho sakta?

A: Ho sakta hai, lekin quality bahut gir jayegi. Reasons:

1. **Separation of Concerns:** Har agent ek specialized task karta hai. Ek LLM se “search karo, analyze karo, verify karo, likho” kehna → mediocre output at every stage.

2. **Different Temperature Settings:** Researcher (0.0 — deterministic), Analyst (0.1 — controlled), Fact Checker (0.0 — strict), Writer (0.3 — creative). Ek LLM call mein ye flexibility nahi milti.
3. **Feedback Loop:** 4 agents ke saath Fact Checker independently decide karta hai ki quality sufficient hai ya nahi. Monolithic approach mein ye “self-check” reliable nahi hota (LLM apne hi mistakes identify nahi kar pata).
4. **Debuggability:** Agar report mein koi issue hai, toh pinpoint kar sakte ho — “Analyst ne ye theme miss kiya” ya “Researcher ne ye source nahi dhundha.”
5. **Scalability:** Future mein naye agents add kar sakte ho (e.g., Translator, Summarizer) bina existing code change kiye.



Q3: TypedDict kyun use kiya? Pydantic BaseModel kyun nahi?

A: LangGraph specifically TypedDict ke saath optimized hai kyunki:

1. **Performance:** TypedDict runtime pe sirf ek regular dict hai — no validation overhead. Pydantic BaseModel har field pe validation run karta hai (slow for frequent state updates).
2. **Reducer Support:** LangGraph ka Annotated[list, operator.add] pattern TypedDict ke saath kaam karta hai. Pydantic mein ye implement karna extra effort hai.
3. **Simplicity:** State ek simple key-value store hai — Pydantic ki advanced features (validators, serialization) yahan overkill hain.
4. **LangGraph Convention:** LangGraph documentation aur examples TypedDict recommend karti hai. Community support aur compatibility better hai.

Kab Pydantic use karein? Agar input validation chahiye (e.g., “topic 500 characters se zyada na ho”), toh Pydantic layer add kar sakte ho **state ke bahar** — e.g., Streamlit input validation mein.



Q4: operator.add reducer kaise internally kaam karta hai?

A: Jab LangGraph node ka output state mein merge karta hai:

```

1  # Internally (simplified):
2  def merge_state(current_state, node_output):
3      for key, new_value in node_output.items():
4          if has_reducer(key):
5              reducer = get_reducer(key) # operator.add
6              current_state[key] = reducer(
7                  current_state[key], new_value
8              )
9              # operator.add([old], [new]) = [old] + [new]
10         else:
11             current_state[key] = new_value # overwrite
12     return current_state

```

operator.add Python ka built-in hai. Lists ke liye: operator.add([1,2], [3,4]) = [1,2,3,4]. Ye list concatenation hai.

Custom reducer bhi bana sakte ho:

```

1  def keep_latest_n(old, new, n=10):

```

```

2     """Keep only last n items."""
3     combined = old + new
4     return combined[-n:]
5
6 messages: Annotated[list[str], keep_latest_n]

```

👤 Q5: Feedback loop mein infinite recursion kaise prevent kiya?

A: Triple safety net hai:

1. **revision_count cap:** Fact Checker mein:

```

1 if "NEEDS_REVISION" in result.upper():
2     if revision_count < MAX_REVISIONS: # MAX = 2
3         passed = False # Only loop if revisions remain

```

2 revisions ke baad chahe kuch bhi ho, passed = True force ho jata hai.

2. **LangGraph recursion_limit:**

```

1 result = graph.invoke(state, {"recursion_limit": 25})

```

Max 25 node executions. Agar kisi bug se loop nahi rukta, toh LangGraph exception throw karega.

3. **Default True:** state.get("fact_check_passed", True) — agar key missing ho (unexpected edge case), toh assume passed.

👤 Q6: Tavily vs Google Search API — kyun Tavily choose kiya?

A:

	Tavily	Google Search API
Output	Structured (title, content, URL)	Snippets only
AI Optimized	Yes (content extraction)	No (raw HTML)
Depth Modes	Basic + Advanced	One mode
Free Tier	1000 searches/month	100 queries/day
LangChain	First-class integration	Community wrapper
Content	Full article extraction	Title + snippet

Tavily specifically LLM applications ke liye designed hai — ye web pages ka content extract karke structured JSON deta hai, jo directly LLM ke prompt mein jaata hai. Google API se raw HTML parse karna padta — extra code aur error-prone.

👤 Q7: Groq kyun use kiya? OpenAI ya Anthropic kyun nahi?

A: Speed + Cost:

- **Groq:** Custom LPU hardware pe runs — LLaMA 3.3 70B model ko ~300 tokens/second pe serve karta hai. Free tier available.
- **OpenAI (GPT-4):** Better quality, but \$30/1M input tokens. Humara pipeline 6+ LLM calls karta hai per session — cost quickly adds up.
- **Anthropic (Claude):** Comparable quality, similar pricing to OpenAI.

Trade-off: LLaMA 3.3 70B ki quality GPT-4 se thodi kam hai, lekin “good enough” for research reports. And with Groq’s speed, user experience significantly better hai

— report 30 seconds mein ready vs 2 minutes.

Flexibility: MODEL_NAME configurable hai — .env mein change karo, any Groq-supported model use karo (Mixtral, Gemma, etc.)

Q8: System Prompts kitne important hain? Kaise design kiye?

A: System prompts **game changer** hain. Bina proper system prompt ke LLM generic output dega.

Design Principles:

1. **Role Definition:** “You are an expert research analyst” — LLM ko persona do.
2. **Structured Output:** “You MUST include: 1) Key Themes, 2) Contradictions...” — exact format batao.
3. **Constraints:** “Do NOT make up information” — hallucination prevent karo.
4. **Output Format:** “End with VERDICT: PASSED or NEEDS_REVISION” — parseable output ensure karo.

Example Impact:

```
1 # Bad prompt:
2 "Analyze this data."
3 # Output: Random, inconsistent format
4
5 # Good prompt (humara):
6 "You are an expert research analyst. You MUST:
7 1. Identify 3-5 KEY THEMES
8 2. Note CONTRADICTIONS..."
9 # Output: Structured, consistent, high-quality
```

Q9: Agar ek agent fail ho jaye toh kya hoga?

A: Har agent mein defensive programming hai:

1. **Researcher:** Web search fail → error message append karke continue. RAG fail → skip RAG, web results use karo. No queries generated → fallback to topic itself.
2. **Analyst:** No research data → “No data available” return, pipeline continues.
3. **Fact Checker:** No analysis → auto-pass (proceed to Writer).
4. **Writer:** If analysis/data missing → still generates report with whatever is available.
5. **Pipeline Level:** try/except wrapper in app.py catches any unhandled exceptions and displays error to user.

Key Principle: “Degrade gracefully, never crash silently.” User ko hamesha pata chale ki kya hua.

Q10: Project ki scalability discuss karo.

A:

Current Bottlenecks:

- Groq rate limits (30 RPM free tier)
- Single-threaded Streamlit

- In-process ChromaDB

Scaling Strategies:

1. **Horizontal:** Streamlit ko multiple instances mein run karo behind a load balancer (Nginx/Traefik).
2. **LLM:** Groq paid tier (→ higher rate limits) ya multiple LLM providers rotate karo.
3. **Vector DB:** ChromaDB ko dedicated server pe deploy karo (`chroma run -host 0.0.0.0`) ya Pinecone/Weaviate cloud use karo.
4. **Async:** LangGraph async support hai — `await graph.ainvoke()` se I/O bound tasks parallel ho sakte hain.
5. **Caching:** Frequent queries ke results cache karo (Redis) — same topic pe repeat research avoid.

14.2 LangGraph & Orchestration (Q11–Q20)

Q11: LangGraph vs LangChain Agents — kya fark hai?

A:

- **LangChain Agents:** Single LLM jo tools call karta hai in a ReAct loop. LLM decide karta hai ki kab kaunsa tool use kare. Non-deterministic — same input pe different tool sequences.
- **LangGraph:** Explicit graph with defined nodes and edges. Developer control karta hai flow. Deterministic (except conditional edges based on LLM output). Better for multi-agent systems.

Analogy: LangChain Agent = Ek employee jo apne se decide kare kya karna hai. LangGraph = Assembly line with specific stations — har station pe specific kaam.

Q12: StateGraph vs MessageGraph — kab kya use karein?

A:

- **StateGraph:** Custom state schema (TypedDict) define karo. Multiple fields, reducers, complex data flow. Humara project ye use karta hai.
- **MessageGraph:** State sirf messages list hai (`list[BaseMessage]`). Simple chatbot-style applications ke liye. No custom fields.

Humein **StateGraph** chahiye kyunki humara state complex hai — `topic`, `research_data`, `analysis`, `fact_check_passed`, etc. **MessageGraph** mein ye sab manage karna mushkil hota.

Q13: `compile()` kya karta hai internally?

A: `workflow.compile()` ke dauran:

1. **Validation:** Check karta hai ki saare edges valid nodes ko point kar rahe hain. “Dangling edges” detect karta hai.
2. **Cycle Detection:** Agar unbounded cycle hai (no revision limit), toh warning deta hai.

3. **Entry/Exit Points:** Verify karta hai ki entry point defined hai aur END reachable hai.
4. **Optimization:** Internally ek efficient execution plan banata hai.
5. **Immutability:** Compiled graph mein naye nodes/edges add nahi kar sakte (frozen).

Compile ke baad `.invoke()`, `.stream()`, `.ainvoke()`, `.astream()` methods available hote hain.

 **Q14:** Conditional edges ke alawa aur kaunse edge types hain LangGraph mein?

A:

1. **Normal Edge:** `add_edge(A, B)` — A ke baad hamesha B
2. **Conditional Edge:** `add_conditional_edges(A, func, map)` — Function decide kare
3. **Entry Edge:** `set_entry_point(A)` — Graph ka starting node
4. **Terminal Edge:** `add_edge(A, END)` — Graph end karo
5. **Branching:** Conditional edge se multiple paths possible

 **Q15:** Agar 6th agent add karna ho (e.g., Translator), toh kya changes karne padenge?

A: 4 simple steps:

1. **Agent banao:** `agents/translator.py` — function `translator_node(state)`
2. **State update:** `schema.py` mein `translated_report: str` field add karo
3. **Graph update:** `workflow.py` mein:

```
1 workflow.add_node("translator", translator_node)
2 workflow.add_edge("writer", "translator")
3 workflow.add_edge("translator", END)
4 # Remove: workflow.add_edge("writer", END)
```

4. **UI update:** `app.py` mein translator status pill add karo
Existing agents ka code change nahi hota — ye **Open/Closed Principle** hai (open for extension, closed for modification).

 **Q16:** LangGraph mein checkpointing kya hai?

A: Checkpointing LangGraph mein state persistence hai. Har node execution ke baad state save hota hai, taki:

- App crash ho toh resume kar sako from last checkpoint
- Time travel — kisi bhi previous state pe jaao
- Human-in-the-loop — pause karo, human input lo, resume karo

```
1 from langgraph.checkpoint.sqlite import SqliteSaver
2
3 checkpointer = SqliteSaver.from_conn_string(":memory:")
4 graph = workflow.compile(checkpointer=checkpointer)
5
6 # Run with thread_id for state persistence
```

```

7 config = {"configurable": {"thread_id": "user-123"}}
8 result = graph.invoke(state, config)

```

Humara project currently checkpointing use nahi karta (simplicity ke liye), lekin production mein zaroori hai.

 **Q17:** stream() method ka internal mechanism explain karo.

A: stream() ek Python generator hai:

```

1 # Simplified internal implementation:
2 def stream(self, input_state, config):
3     state = input_state.copy()
4     current_node = self.entry_point
5
6     while current_node != END:
7         node_fn = self.nodes[current_node]
8         output = node_fn(state)           # Execute node
9         state = merge(state, output)      # Merge with reducers
10        yield {current_node: output}      # ←- YIELD here!
11        current_node = self.get_next_node(
12            current_node, state
13        )

```

Har yield pe caller ko control milta hai — Streamlit UI update kar sakta hai. Generator lazy evaluation use karta hai — next node tab tak execute nahi hota jab tak caller next() call na kare (ya for loop continue kare).

 **Q18:** LangGraph mein parallel execution possible hai?

A: Haan! Agar do nodes mein dependency nahi hai:

```

1 # Parallel execution example:
2 workflow.add_edge("researcher", "analyst_1")
3 workflow.add_edge("researcher", "analyst_2")
4 # analyst_1 and analyst_2 run in parallel!
5
6 # Then merge:
7 workflow.add_edge("analyst_1", "merger")
8 workflow.add_edge("analyst_2", "merger")

```

Humara project sequential hai (analyst → fact_checker), lekin future mein multiple analysts parallel run kar sakte hain (e.g., one for themes, one for contradictions).

 **Q19:** LangGraph mein error handling best practices kya hain?

A:

1. **Node-level try/except:** Har agent mein exceptions catch karo aur meaningful error messages return karo.
2. **Fallback nodes:** add_conditional_edges se error state pe fallback node route karo.

3. **State error field:** Humara error: `str` field — koi bhi agent yahan error store kar sakta hai.
4. **Timeout:** LangGraph mein per-node timeout set kar sakte ho.
5. **Retry:** LangChain mein `llm.with_retry()` se LLM calls auto-retry hoti hain on transient errors.

 **Q20:** Humara graph ka recursion_limit 25 kyun hai?

A: Worst case calculation:

- Normal flow: 4 nodes ($R \rightarrow A \rightarrow FC \rightarrow W$)
- 1 revision: +3 nodes ($R \rightarrow A \rightarrow FC$)
- 2 revisions: +3 nodes ($R \rightarrow A \rightarrow FC$)
- Total: $4 + 3 + 3 = 10$ nodes max

25 is 2.5x safety margin. Agar kisi bug se loop expected se zyada chale, toh bhi 25 pe rok jayega. Too high limit (e.g., 1000) dangerous hai — infinite loop mein API costs badh jayenge.

14.3 RAG & Vector Databases (Q21–Q30)

 **Q21:** RAG pipeline step-by-step explain karo.

A:

1. Ingestion Phase:

- PDF $\xrightarrow{\text{PyPDFLoader}}$ Text (page-wise)
- Text $\xrightarrow{\text{RecursiveCharacterTextSplitter}}$ Chunks (1000 chars, 200 overlap)
- Chunks $\xrightarrow{\text{HuggingFace all-MiniLM}}$ Vectors (384-dim)
- Vectors $\xrightarrow{\text{ChromaDB}}$ Persistent storage

2. Retrieval Phase:

- User query $\xrightarrow{\text{Embedding}}$ Query vector
- Query vector $\xrightarrow{\text{Cosine similarity}}$ Top-k similar chunks
- Similar chunks $\xrightarrow{\text{Format}}$ Context with source metadata

3. Augmentation Phase:

- Retrieved chunks + Web search results $\rightarrow$ Combined context
- Combined context + System prompt $\rightarrow$ LLM prompt

4. Generation Phase:

- LLM generates response using augmented context
- Response is grounded in actual documents $\rightarrow$ less hallucination

 **Q22:** Chunk size 1000 aur overlap 200 kyun?

A:

- **Chunk size 1000:** Sweet spot hai. Too small (100) $\rightarrow$ context lost, too many chunks. Too large (5000) $\rightarrow$ embedding quality drops (model dilutes information), retrieval precision kam.

- **Overlap 200 (20%):** Sentences chunk boundary pe split hone ka risk minimize. 20% standard practice hai. Too much overlap (500) → redundancy aur storage waste.

Practical Impact:

```

1 # 10-page PDF (~30,000 chars) ke saath:
2 # Chunk size 1000, overlap 200:
3 # ~37 chunks banengi (with overlap)
4 # Bina overlap: ~30 chunks

```

👤 Q23: RecursiveCharacterTextSplitter vs aur splitters?

A:

Splitter	Use Case
CharacterTextSplitter	Simple split at fixed positions
RecursiveCharacterTextSplitter	Best general purpose. Tries multiple separators hierarchically.
TokenTextSplitter	Token-based (for precise LLM token limits)
MarkdownHeaderTextSplitter	Markdown documents (split by headers)
HTMLHeaderTextSplitter	HTML documents

“Recursive” isliye best hai kyunki ye **semantic boundaries** respect karta hai — pehle paragraph break pe split karne ki koshish, phir sentence, phir word. Ye chunk ka “meaning” preserve karta hai.

👤 Q24: ChromaDB vs Pinecone vs FAISS — comparison?

A:

	ChromaDB	Pinecone	FAISS
Type	Embedded DB	Cloud SaaS	Library
Cost	Free	Paid	Free
Persistence	Disk	Cloud	Manual
Setup	pip install	API signup	pip install
Scalability	Moderate	High	High (RAM)
Best For	Prototypes/small	Production	Research

Humne ChromaDB choose kiya kyunki: free, easy setup (pip install), persistent storage (restart pe data safe), aur LangChain integration first-class.

👤 Q25: Embedding model all-MiniLM-L6-v2 kyun?

A:

- **Free:** No API cost. HuggingFace se download, locally run.
- **Small:** ~80MB. CPU pe fast (14K sentences/sec).
- **Quality:** Sentence-BERT family. Semantic similarity tasks mein excellent.
- **Dimension:** 384 — compact yet effective. OpenAI ada-002 1536 dimensions use karta hai (better quality but API cost).

Upgrade Path: Better quality chahiye? `.env` mein change karo:

```

1 EMBEDDING_MODEL=BAAI/bge-large-en-v1.5 # 1024 dim, better
  quality

```

 Q26: Cosine similarity vs Euclidean distance — kab kya?**A:**

- **Cosine Similarity:** Direction compare karta hai, magnitude nahi. “AI is great” aur “AI IS GREAT!!!” similar honge (same direction, different magnitude). Text search ke liye best.
- **Euclidean Distance:** Straight-line distance. Magnitude matters. Image search, anomaly detection ke liye better.

Humne `normalize_embeddings=True` set kiya — normalized vectors ke liye cosine similarity = dot product (faster computation). ChromaDB default mein cosine similarity use karta hai.

 Q27: Duplicate documents kaise handle kiye?**A:** MD5 content hashing:

```

1 content_hash = hashlib.md5(
2     chunk.page_content.encode()
3 ).hexdigest()
4 ids.append(content_hash)

```

Same content → same MD5 hash → same ID. ChromaDB mein same ID wala document overwrite hota hai. Toh:

- Same PDF dobara upload? No duplicates.
- Slightly modified PDF? Naye chunks create honge (different content → different hash).

Note: MD5 cryptographically weak hai (collision attacks), lekin deduplication ke liye sufficient. Security-critical nahi hai.

 Q28: Memory system (past research) ka design explain karo.**A: Two-collection architecture:**

1. `research_docs` — User uploaded PDFs (explicit knowledge)
2. `past_research` — Auto-saved research sessions (implicit memory)

Flow:

1. User researches “AI Ethics” → Writer stores report + data in `past_research`
2. User researches “AI Regulations” → Researcher queries `past_research` → “AI Ethics” context automatically retrieved
3. System “remembers” past work!

Why separate collections? User might want to clear uploaded docs without losing research memory (or vice versa).

 Q29: `similarity_search` vs `max_marginal_relevance_search`?**A:**

- **`similarity_search`:** Top-k most similar chunks return karta hai. Problem: sab chunks similar ho sakte hain (redundancy).
- **MMR (Max Marginal Relevance):** Similar chunks return karta hai but diversity bhi ensure karta hai. Ye relevant + diverse results deta hai.

```

1 # MMR example:
2 results = vectorstore.max_marginal_relevance_search(
3     query, k=5, fetch_k=20, lambda_mult=0.5
4 )
5 # fetch_k=20: 20 candidates fetch karo
6 # lambda_mult=0.5: balance between relevance and diversity
7 # k=5: final 5 results return karo

```

Humara project `similarity_search` use karta hai (simpler, usually sufficient). MMR upgrade hone pe diversity improve hogi.

Q30: RAG ke alternatives kya hain?

A:

1. **Fine-tuning:** LLM ko custom data pe train karo. Permanent knowledge injection. Expensive, slow to update.
2. **In-context Learning:** Pura document prompt mein daal do. No vector DB needed. But context window limit.
3. **Knowledge Graphs:** Structured relationships store karo (entity → relation → entity). Better for structured queries.
4. **Hybrid (RAG + Fine-tuning):** Domain-specific fine-tuned model + RAG for latest data. Best quality, highest cost.

RAG is best balance of quality, cost, and flexibility for our use case.

14.4 MCP & Integration (Q31–Q37)

Q31: MCP Protocol kya hai aur kyun zaroori hai?

A: MCP (Model Context Protocol) Anthropic ne banaya hai. Ye ek **open standard** hai jo AI models ko external tools aur data sources se connect karta hai.

Problem it solves: Pehle har AI tool ka apna custom integration hota tha. Claude ke liye alag, ChatGPT ke liye alag, Cursor ke liye alag. MCP ek “USB for AI” hai — ek standard plug jo sabke saath kaam kare.

Core Concepts:

- **Tools:** Functions jo actions perform karti hain (e.g., `research_topic`)
- **Resources:** Read-only data endpoints (e.g., system info)
- **Prompts:** Pre-defined prompt templates
- **Transports:** Communication channels (stdio, SSE)

Q32: FastMCP vs raw MCP SDK — kyun FastMCP?

A: FastMCP ek **high-level wrapper** hai MCP SDK ke upar:

```

1 # Raw MCP SDK (verbose):
2 from mcp.server import Server
3 server = Server("name")
4

```

```

5 @server.list_tools()
6 async def list_tools():
7     return [Tool(name="...", description="...",
8                 inputSchema={...})]
9
10 @server.call_tool()
11 async def call_tool(name, arguments):
12     if name == "research_topic":
13         return result
14
15 # FastMCP (clean):
16 from fastmcp import FastMCP
17 mcp = FastMCP("name")
18
19 @mcp.tool()
20 def research_topic(topic: str) → str:
21     return result # Schema auto-generated!

```

FastMCP automatically function signatures se JSON schema generate karta hai, docstrings se descriptions, aur type hints se input validation. 80% less boilerplate!

Q33: MCP mein stdio transport kaise kaam karta hai?

A:

1. Client (e.g., Claude Desktop) MCP server ko **subprocess** ke roop mein start karta hai: `python mcp_server.py`
2. Communication **stdin/stdout** se hoti hai (JSON-RPC protocol)
3. Client tool list request bhejta hai → Server tool schemas return karta hai
4. Client specific tool call karta hai → Server execute karke result return karta hai
5. Connection tab tak rehti hai jab tak client running hai

Kab SSE use karein?

- **stdio**: Local tools, desktop applications. Low latency.
- **sse**: Remote servers, web applications. Works over HTTP.

Q34: MCP server test kaise karo?

A:

```

# Method 1: MCP Inspector (visual debugger)
fastmcp dev mcp_server.py
# Browser mein opens -- tools test karo visually

# Method 2: Direct Python test
from mcp_server import research_topic
result = research_topic("AI in healthcare")
print(result)

# Method 3: Claude Desktop integration
# claude_desktop_config.json mein add karo

```

👤 Q35: MCP resources vs tools mein kya fark hai?

A:

- **Tools:** Actions perform karti hain. Side effects ho sakte hain (research execute, documents ingest, KB clear). Client explicitly invoke karta hai.
- **Resources:** Read-only data provide karte hain. No side effects. URI-based addressing (`info://orchestrator`). Client browse kar sakta hai.

Analogy: Tools = Remote control buttons (change something). Resources = Dashboard display (show information).

👤 Q36: MCP server mein error handling kaise karo?

A: Har tool function mein try/except with meaningful error messages:

```

1 @mcp.tool()
2 def research_topic(topic: str) → str:
3     try:
4         result = research_graph.invoke(...)
5         return result.get("final_report", "No report.")
6     except Exception as e:
7         return f"Error: {str(e)}"
8         # Client ko readable error milta hai

```

MCP protocol mein bhi error response format defined hai, lekin string return karna simpler hai aur client ko directly display ho jaata hai.

👤 Q37: Kya MCP server mein authentication add kar sakte ho?

A: Current MCP spec mein authentication built-in nahi hai (stdio local hai toh zaroorat nahi). But SSE transport ke saath:

- **Bearer Token:** HTTP headers mein API key
- **OAuth 2.0:** Full auth flow
- **API Gateway:** Nginx/Traefik se rate limiting + auth add karo

Production deployment mein authentication zaroori hai, especially agar MCP server public network pe expose ho.

14.5 Production & Optimization (Q38–Q50)

👤 Q38: Production mein deploy kaise karenge?

A:

```

# Docker deployment:
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .

```

```
EXPOSE 8501
CMD ["streamlit", "run", "app.py",
     "--server.port=8501",
     "--server.address=0.0.0.0"]
```

- **Docker:** Containerize karo, kahi bhi deploy karo (AWS, GCP, Azure)
- **Streamlit Cloud:** Free hosting for public apps
- **Railway/Render:** Easy PaaS deployment
- **Kubernetes:** Large-scale, auto-scaling deployment

Q39: Caching strategy kya hogi?

A:

1. **LLM Response Cache:** Same prompt pe same response — Redis mein cache karo. LangChain mein `llm = ChatGroq(...).with_cache()` available hai.
2. **Search Results Cache:** Tavily results ko TTL-based cache mein rakho (e.g., 1 hour). Same query pe repeated API calls save.
3. **Embedding Cache:** HuggingFace model already singleton hai (memory cache). Disk cache se startup bhi fast hoga.
4. **Report Cache:** Final reports ko filesystem ya database mein store karo. Same topic pe instant retrieval.

Q40: Monitoring aur logging kaise karenge production mein?

A:

1. **LangSmith:** LangChain ka official tracing tool. Har LLM call, token usage, latency automatically track hota hai.
2. **Python Logging:** `import logging` se structured logs.
3. **Prometheus + Grafana:** Metrics dashboard (response times, error rates).
4. **Sentry:** Error tracking with stack traces.

```
1 # LangSmith setup:
2 import os
3 os.environ["LANGCHAIN_TRACING_V2"] = "true"
4 os.environ["LANGCHAIN_API_KEY"] = "ls_..."
5 # Now all LLM calls are auto-traced!
```

Q41: Rate limiting kaise handle karenge?

A: Groq free tier: 30 requests/minute. Handling strategies:

1. **Exponential Backoff:** Rate limit hit → wait 1s, retry. Fail again → wait 2s, 4s, 8s...
2. **Queue System:** Requests ko queue mein rakho, controlled rate pe process karo.
3. **Multiple API Keys:** Load balance across multiple Groq accounts.
4. **LangChain built-in:** `llm.with_retry(stop_after_attempt=3)` automatic retry with backoff.

 Q42: Testing strategy kya hogi?

A:

1. **Unit Tests:** Har agent function ko mock LLM se test karo.

```
1 from unittest.mock import patch
2
3 @patch('agents.researcher.llm')
4 def test_researcher(mock_llm):
5     mock_llm.invoke.return_value.content = \
6         "query1\nquery2\nquery3"
7     result = researcher_node(test_state)
8     assert "research_data" in result
```

2. **Integration Tests:** Full pipeline run karo with test topic.
3. **RAG Tests:** Document ingest → retrieve → verify content matches.
4. **MCP Tests:** FastMCP inspector se tool testing.

 Q43: Security considerations kya hain?

A:

1. **API Keys:** .env file, never in code. .gitignore mein add.
2. **Prompt Injection:** User topic malicious prompt contain kar sakta hai. Mitigation: input sanitization + system prompt hardening.
3. **PDF Upload:** Malicious PDFs possible. PyPDFLoader safe hai (no JS execution), lekin file size limits lagao.
4. **ChromaDB:** Local persistent directory — access control ensure karo.
5. **MCP:** stdio transport local hai (safe). SSE pe expose karna risky bina auth ke.

 Q44: Agar LLM hallucinate kare toh?

A: Multiple layers of mitigation:

1. **Grounding:** Fact Checker source data ke against verify karta hai
2. **System Prompts:** “Do NOT make up information” explicit instruction
3. **RAG:** Retrieved context provide karna hallucination reduce karta hai (LLM ko “answers” mil jaate hain)
4. **Low Temperature:** 0.0-0.1 for factual agents (less randomness = less hallucination)
5. **Feedback Loop:** Agar hallucinated content Fact Check fail kare, toh re-research hoga

 Q45: Streamlit ki limitations kya hain?

A:

1. **Re-run model:** Har interaction pe pura script re-run — complex state management mushkil
2. **Single-threaded:** Concurrent users ke liye scale nahi karta natively
3. **Limited customization:** CSS injection possible but limited
4. **No WebSocket:** Real-time updates ke liye workarounds chahiye
5. **No auth built-in:** Custom authentication implement karna padta hai

Alternatives: Gradio (ML-focused), FastAPI + React (full control), Chainlit (chat-focused).

👤 Q46: Performance optimization tips?

A:

1. **Embedding Caching:** `@st.cache_resource` se model ek baar load karo
2. **Async LLM Calls:** `ainvoke()` se I/O overlap karo
3. **Batch Embeddings:** Multiple chunks ko ek baar mein embed karo (batch processing)
4. **Streaming:** LLM response stream karo token-by-token for better UX
5. **Compression:** ChromaDB mein vectors compress karo (PQ compression)

👤 Q47: Is project mein kya improvements kar sakte ho?

A:

1. **Multi-format RAG:** PDFs ke alawa Word, PPT, URLs support
2. **Hybrid Search:** Semantic + keyword search combine karo (BM25 + embeddings)
3. **User Feedback:** Report pe thumbs up/down → quality improve over time
4. **Multi-language:** Translator agent add karo
5. **Visualization:** Research data ka automated chart/graph generation
6. **API Endpoint:** FastAPI wrapper for programmatic access
7. **Collaborative:** Multiple users same research pe work karein
8. **Citations:** Academic citation format support (APA, MLA)

👤 Q48: LangGraph vs CrewAI vs AutoGen — comparison?

A:

	LangGraph	CrewAI	AutoGen
Paradigm	Graph-based	Role-based	Conversation-based
Control	Developer-defined flow	AI-decided tasks	Agent negotiation
Determinism	High	Medium	Low
Complexity	Medium	Low	High
Best For	Pipelines	Team simulation	Research/debate

Humne LangGraph choose kiya kyunki humein **predictable, deterministic pipeline** chahiye tha with explicit control over agent flow.

👤 Q49: Agar ye project resume pe daal rahe ho toh kaise describe karoge?

A: Bullet points for resume:

Multi-Agent Research Orchestrator | Python, LangGraph, Groq, ChromaDB

- Engineered a 4-agent AI pipeline (Researcher, Analyst, Fact-Checker, Writer) using LangGraph StateGraph with conditional feedback loops for self-correcting research
- Implemented RAG system with ChromaDB + HuggingFace embeddings for persistent knowledge base with PDF ingestion and cross-session memory

- Built MCP server (FastMCP) enabling integration with Claude Desktop and Cursor IDE for tool-augmented AI workflows
- Designed premium Streamlit UI with glassmorphism theme, real-time agent status tracking, and streaming pipeline execution
- Achieved zero-cost operation using Groq (LLaMA 3.3 70B) + Tavily free tiers with local HuggingFace embeddings

 Q50: Is project ka future scope kya hai?

A:

1. **Agent Memory Evolution:** Episodic memory (specific sessions) + Semantic memory (general knowledge) + Procedural memory (learned strategies) — 3-tier memory architecture.
2. **Multi-Modal Research:** Images, videos, charts bhi analyze karo — vision LLMs integrate karo.
3. **Collaborative Research:** Multiple users ek saath research karein, shared knowledge base.
4. **Auto-citation:** Academic papers detect karo aur proper citation format mein add karo.
5. **Research Graphs:** Visualize connections between topics, findings, and sources as knowledge graphs.
6. **Self-Improvement:** Agent performance track karo, low-quality outputs pe automatic prompt refinement.
7. **Domain Specialization:** Medical research, legal research, financial research — domain-specific agents with specialized tools.
8. **Real-time Updates:** RSS feeds, news APIs se continuous research updates.

15 Quick Reference Cheat Sheets

15.1 LangGraph Cheat Sheet

```
1 # 1. Basic Graph Setup
2 from langgraph.graph import StateGraph, END
3 workflow = StateGraph(MyState)
4 workflow.add_node("name", function)
5 workflow.set_entry_point("name")
6 workflow.add_edge("A", "B")
7 workflow.add_edge("B", END)
8 graph = workflow.compile()
9
10 # 2. Conditional Routing
11 def router(state):
12     if state["condition"]:
13         return "path_a"
14     return "path_b"
15
16 workflow.add_conditional_edges(
17     "source", router,
18     {"path_a": "node_a", "path_b": "node_b"}
19 )
20
21 # 3. State with Reducers
22 from typing import Annotated, TypedDict
23 import operator
24
25 class MyState(TypedDict):
26     data: Annotated[list[str], operator.add]
27     # Append mode
28
29 # 4. Execution
30 result = graph.invoke(initial_state)
31 for event in graph.stream(initial_state):
32     print(event)
```

🔗 LangGraph Patterns

15.2 RAG Cheat Sheet

```
1 # 1. Embeddings
2 from langchain_huggingface import HuggingFaceEmbeddings
3 embeddings = HuggingFaceEmbeddings(
4     model_name="sentence-transformers/all-MiniLM-L6-v2"
5 )
```

```
6
7 # 2. Vector Store
8 from langchain_chroma import Chroma
9 vectorstore = Chroma(
10     collection_name="my_docs",
11     embedding_function=embeddings,
12     persist_directory="./chroma_db"
13 )
14
15 # 3. Ingest Documents
16 from langchain_community.document_loaders import PyPDFLoader
17 from langchain_text_splitters import (
18     RecursiveCharacterTextSplitter
19 )
20
21 loader = PyPDFLoader("file.pdf")
22 docs = loader.load()
23 splitter = RecursiveCharacterTextSplitter(
24     chunk_size=1000, chunk_overlap=200
25 )
26 chunks = splitter.split_documents(docs)
27 vectorstore.add_documents(chunks)
28
29 # 4. Retrieve
30 results = vectorstore.similarity_search(
31     "my query", k=5
32 )
33 for doc in results:
34     print(doc.page_content)
35     print(doc.metadata)
```

🔗 RAG Patterns

15.3 MCP Cheat Sheet

```
1 # 1. Server Setup
2 from fastmcp import FastMCP
3 mcp = FastMCP("MyServer", description="...")
4
5 # 2. Define Tool
6 @mcp.tool()
7 def my_tool(param: str) → str:
8     """Tool description (becomes MCP schema)."""
9     return f"Result: {param}"
10
11 # 3. Define Resource
12 @mcp.resource("info://my-resource")
13 def my_resource() → str:
14     """Read-only data."""
15     return "Resource content"
```

```
16
17 # 4. Run Server
18 if __name__ == "__main__":
19     mcp.run(transport="stdio")
20
21 # 5. Test with Inspector
22 # Terminal: fastmcp dev my_server.py
23
24 # 6. Claude Desktop Config
25 # {
26 #     "mcpServers": {
27 #         "myserver": {
28 #             "command": "python",
29 #             "args": ["path/to/my_server.py"]
30 #         }
31 #     }
32 # }
```

 MCP Patterns

15.4 Streamlit Cheat Sheet

```
1 import streamlit as st
2
3 # 1. Page Config (MUST be first!)
4 st.set_page_config(page_title="App", layout="wide")
5
6 # 2. Session State
7 if "counter" not in st.session_state:
8     st.session_state.counter = 0
9
10 # 3. Layout
11 col1, col2 = st.columns(2)
12 with col1:
13     st.write("Left column")
14
15 # 4. Input Widgets
16 text = st.text_input("Enter text")
17 btn = st.button("Click me")
18 files = st.file_uploader("Upload", type=["pdf"],
19                           accept_multiple_files=True)
20
21 # 5. Feedback
22 st.success("Done!")
23 st.error("Failed!")
24 st.warning("Careful!")
25 with st.spinner("Loading..."):
26     time.sleep(2)
27
28 # 6. Custom CSS
```

```
29 st.markdown("<style>...</style>",  
30             unsafe_allow_html=True)  
31  
32 # 7. Force Re-render  
33 st.rerun()
```

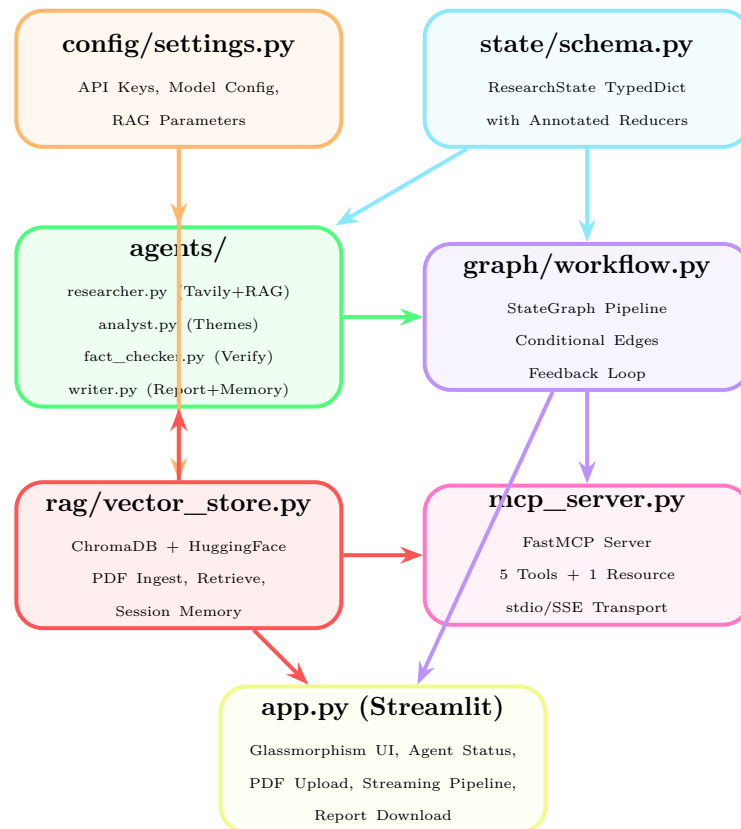
 Streamlit Patterns

16 Glossary — Important Terms

Term	Meaning (Hinglish)
Agent	Ek autonomous AI entity jo specific task karta hai
Annotated	Python typing feature — type ke saath extra meta-data attach karo
API Key	Authentication token — “password” for using an API service
Callback	Function jo kisi event pe automatically call hota hai
ChromaDB	Open-source vector database for embedding storage and search
Chunking	Bade text ko chhote pieces mein todna for processing
Conditional Edge	Graph edge jo runtime pe decide karta hai kahan jaana hai
Context Window	LLM ka maximum input size (tokens mein)
Cosine Similarity	Do vectors ke beech angle-based similarity measure
Embedding	Text ko numerical vector mein convert karna
END	LangGraph ka terminal node — pipeline yahan khatam
FastMCP	Python library for easy MCP server creation
Feedback Loop	Output ko input mein wapas bhejkar improve karna
Generator	Python function jo yield se values lazily produce karta hai
Glassmorphism	UI design trend with frosted glass effect
Groq	AI hardware company providing fast LLM inference
Hallucination	LLM ka false/fabricated information generate karna
HNSW	Hierarchical Navigable Small World — fast ANN algorithm
LangChain	Framework for building LLM-powered applications
LangGraph	LangChain ka extension for multi-agent graph workflows
LLM	Large Language Model (GPT, LLaMA, etc.)
MCP	Model Context Protocol — AI tool interoperability standard
Metadata	Data about data (e.g., file name, page number of a chunk)
Node	LangGraph mein ek processing unit (agent function)
operator.add	Python ka addition operator as a function — list concatenation
Persist	Data ko disk pe save karna (RAM se permanent storage)
Prompt	LLM ko diya gaya input text/instruction
RAG	Retrieval Augmented Generation — search then generate
Reducer	Function jo multiple values ko merge karta hai (e.g., list append)

Semantic Search	Meaning-based search (vs keyword matching)
Session State	Streamlit mein persistent storage across script reruns
Singleton	Design pattern — ek class ka sirf ek instance bane
StateGraph	LangGraph ka graph class with typed state management
Streamlit	Python framework for building web apps quickly
System Prompt	LLM ka “role definition” — constant throughout conversation
Tavily	AI-optimized web search engine API
Temperature	LLM randomness control (0=deterministic, 1=creative)
Token	LLM ka processing unit (~ 4 chars or ~ 0.75 words)
TypedDict	Python dict with defined key types
Vector	Mathematical array of numbers representing data
Vector Store	Database optimized for storing and searching vectors

17 Final Summary — Pura Project Ek Nazar Mein



💡 Yaad Rakhne Ka Tip

Key Takeaways:

1. **Modular Architecture:** Har component independent hai. Koi bhi agent replace/add kar sakte ho bina baaki code change kiye.
2. **Shared State Pattern:** TypedDict with reducers — safe, typed, accumulating data flow.
3. **Feedback Loop:** Self-correcting pipeline with bounded revisions.
4. **RAG Integration:** Documents + Memory — system time ke saath smarter hota hai.
5. **MCP Protocol:** Universal AI tool access — one system, many clients.
6. **Zero Cost:** Free Groq + Tavily + Local Embeddings + ChromaDB.
7. **Production Ready:** Error handling, graceful degradation, configurable via env vars.

“Jo samajh ke padhega, woh interview mein chhaa jayega!”

Notes Generated for Multi-Agent Research Orchestrator Project
August 11, 2026

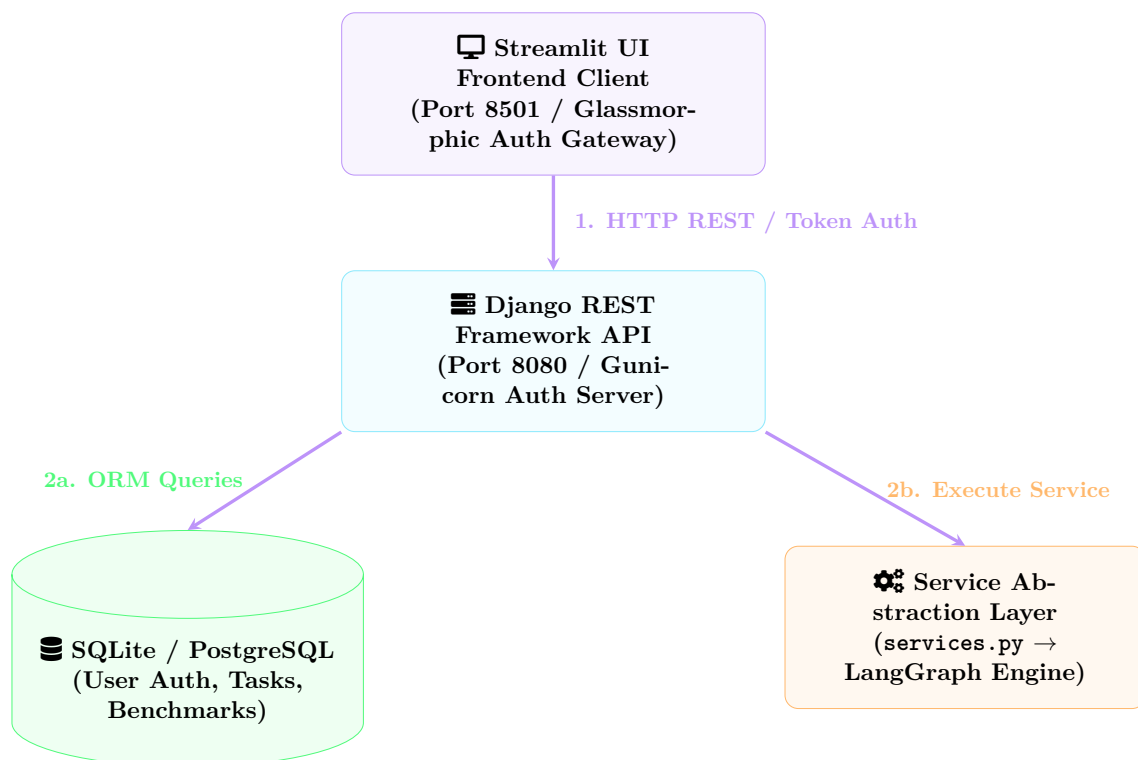
18 Chapter 11: Auth-First Django REST Framework Backend, Port 8080 & System Architecture

💡 Yaad Rakhne Ka Tip

🔗 **Chapter Goal:** Is chapter mein hum Multi-Agent Research Orchestrator ke **Backend Layer** ko deep dive mein samjhenge. Pehle project sirf Client-side Streamlit + LangGraph standalone mode mein chala raha tha. Lekin production-grade enterprise system banane ke liye humne ek **Auth-First Django REST Framework (DRF) Backend** integrate kiya hai jo Port 8080 par chalta hai, Token-based Authentication, Service Layer Abstraction, User-Isolated Database Storage, aur Docker Containerization provide karta hai.

18.1 11.1 Backend Architecture Overview: Why Django DRF?

Humne system ko **Microservice-style Architecture** mein separate kiya hai taaki UI presentation layer, authentication, data persistence, aur AI multi-agent orchestration cleanly decoupled rahein:



18.1.1 Core Backend Architectural Principles:

1. **Auth-First Security Gateway:** Unauthenticated users research pipeline ko run nahi kar sakte. Client sabse pehle `POST /api/v1/auth/login/` request bhej kar DRF Auth Token obtain karta hai, jo har request ke `Authorization: Token <key>` header mein bheja jata hai.
2. **Port 8080 Dedicated Deconfliction:** machine par Port 8000 pehle se kisi aur DRF project (e.g. E-Commerce Platform) se occupied hone ki wajah se `404 Page Not Found` aara ha tha. Humne Orchestrator Backend ko explicitly dedicated `**Port 8080**` (`http://127.0.0.1:8080/api/v1/`) par configure kiya.
3. **Service Layer Abstraction Pattern (`services.py`):** REST API Views ko directly LangGraph workflow engine se tight-couple karne ke bajaye, saari orchestration logic `ResearchOrchestratorService` class ke andar encapsulate ki gayi hai.
4. **Strict Multi-Tenant User Isolation:** Har database query mein `filter(user=request.user)` enforce kiya jata hai. Isse User A kabhi bhi User B ke research tasks ya evaluation benchmark reports ko nahi dekh sakta.

18.2 11.2 Database Schema & Domain Models (backend/api/mo

Backend persistence layer mein do core domain models design kiye gaye hain: `ResearchTask` (research sessions ke liye) aur `BenchmarkResult` (LLM evaluation benchmarks ke liye).

```
1 from django.db import models
2 from django.contrib.auth.models import User
3
4 class ResearchTask(models.Model):
5     STATUS_CHOICES = [
6         ('PENDING', 'Pending'),
7         ('RUNNING', 'Running'),
8         ('COMPLETED', 'Completed'),
9         ('FAILED', 'Failed'),
10    ]
11
12    user = models.ForeignKey(
13        User,
14        on_delete=models.CASCADE,
15        related_name='research_tasks',
16        null=True,
17        blank=True
18    )
19    topic = models.CharField(max_length=500)
20    status = models.CharField(max_length=50, choices=STATUS_CHOICES,
21                             , default='PENDING')
22    final_report = models.TextField(blank=True, null=True)
23    execution_time_seconds = models.FloatField(default=0.0)
24    word_count = models.IntegerField(default=0)
25    created_at = models.DateTimeField(auto_now_add=True)
26
27    class Meta:
28        ordering = ['-created_at']
29
30    def __str__(self):
31        return f"{self.topic[:30]} ({self.status})"
32
33 class BenchmarkResult(models.Model):
34     user = models.ForeignKey(
35         User,
36         on_delete=models.CASCADE,
37         related_name='benchmark_results',
38         null=True,
39         blank=True
40     )
41     topic = models.CharField(max_length=500)
42     single_agent_report = models.TextField()
43     multi_agent_report = models.TextField()
44     single_agent_latency = models.FloatField()
45     multi_agent_latency = models.FloatField()
```

```
46 single_agent_completeness = models.FloatField(default=0.0)
47 multi_agent_completeness = models.FloatField(default=0.0)
48 winner = models.CharField(max_length=50, default='MULTI_AGENT')
49 created_at = models.DateTimeField(auto_now_add=True)
50
51 class Meta:
52     ordering = ['-created_at']
53
54 def __str__(self):
55     return f"Benchmark: {self.topic[:30]} - Winner: {self.winner}"
```

Listing 1: backend/api/models.py — Django Domain Models

18.2.1 Deep Technical Explanation:

- **models.ForeignKey(User, on_delete=models.CASCADE):** User account delete hone par uske saare linked research tasks aur benchmark results automatically database se cascade-delete ho jate hain (Referential Integrity).
- **ordering = ['-created_at']:** Meta ordering parameter default SQL ORDER BY created_at DESC add karta hai, jisse client side par hamesha latest tasks sabse upar dikhte hain.
- **STATUS_CHOICES:** State machine pattern asynchronous workflow lifecycle (PENDING → RUNNING → COMPLETED / FAILED) ko track karta hai.

18.3 11.3 Serializers Layer (backend/api/serializers.py)

DRF Serializers Python objects (ORM QuerySets) ko JSON representation mein convert (serialize) karte hain aur client se aane wale incoming HTTP payloads ko validate (deserialize) karte hain.

```
1 from rest_framework import serializers
2 from django.contrib.auth.models import User
3 from .models import ResearchTask, BenchmarkResult
4
5 class UserSerializer(serializers.ModelSerializer):
6     class Meta:
7         model = User
8         fields = ['id', 'username', 'email', 'first_name', 'last_name']
9
10
11 class RegisterSerializer(serializers.ModelSerializer):
12     password = serializers.CharField(write_only=True, min_length=6)
13
14     class Meta:
15         model = User
16         fields = ['username', 'email', 'password']
17
18     def create(self, validated_data):
19         user = User.objects.create_user(
20             username=validated_data['username'],
21             email=validated_data.get('email', ''),
22             password=validated_data['password']
23         )
24         return user
25
26
27 class ResearchTaskSerializer(serializers.ModelSerializer):
28     user = UserSerializer(read_only=True)
29
30     class Meta:
31         model = ResearchTask
32         fields = '__all__'
33
34
35 class BenchmarkResultSerializer(serializers.ModelSerializer):
36     user = UserSerializer(read_only=True)
37
38     class Meta:
39         model = BenchmarkResult
40         fields = '__all__'
```

Listing 2: backend/api/serializers.py — DRF Data Serializers

18.3.1 Deep Technical Explanation:

- **write_only=True:** Password string ko response output se security reasons ki wajah se omit kar deta hai.
- **User.objects.create_user():** Raw password ko plain-text ke bajaye Django ke default PBKDF2 with SHA256 hashing algorithm se securely hash karke DB mein store karta hai.
- **Nested Serialization (user = UserSerializer(read_only=True)):** Task detail JSON response ke saath user details ko cleanly nest karke return karta hai.

18.4 11.4 Service Layer Abstraction (backend/api/services.py)

Business Logic ko API Views se decouple karne ke liye Service Layer pattern use kiya gaya hai. Isse "Fat Views" Anti-Pattern avoid hota hai.

```

1 import time
2 from typing import Dict, Any, Optional
3 from django.contrib.auth.models import User
4 from .models import ResearchTask, BenchmarkResult
5 from graph.workflow import run_research_workflow
6 from evaluation.evaluator import BenchmarkEvaluator
7
8 class ResearchOrchestratorService:
9     # Service handling orchestration task execution and history
10    # recording.
11
12    @staticmethod
13    def execute_research_task(topic: str, user: Optional[User] =
14    None) → Dict[str, Any]:
15        task = ResearchTask.objects.create(
16            topic=topic,
17            user=user,
18            status='RUNNING'
19        )
20        start_time = time.time()
21        try:
22            result = run_research_workflow(topic)
23            elapsed = time.time() - start_time
24            report = result.get('final_report', '')
25
26            task.final_report = report
27            task.status = 'COMPLETED'
28            task.execution_time_seconds = round(elapsed, 2)
29            task.word_count = len(report.split())
30            task.save()
31
32            return {
33                'task_id': task.id,
34                'status': 'COMPLETED',
35                'report': report,
36                'execution_time': task.execution_time_seconds,
37                'word_count': task.word_count
38            }
39        except Exception as e:
40            task.status = 'FAILED'
41            task.save()
42            raise e

```

Listing 3: backend/api/services.py — Business Logic Layer

18.5 11.5 API Endpoints & Token Authentication (backend/api

```
1 from rest_framework import status
2 from rest_framework.views import APIView
3 from rest_framework.response import Response
4 from rest_framework.permissions import AllowAny, IsAuthenticated
5 from rest_framework.authtoken.models import Token
6 from django.contrib.auth import authenticate
7 from .models import ResearchTask
8 from .serializers import UserSerializer, RegisterSerializer,
   ResearchTaskSerializer
9 from .services import ResearchOrchestratorService
10
11 class RegisterView(APIView):
12     permission_classes = [AllowAny]
13
14     def post(self, request):
15         serializer = RegisterSerializer(data=request.data)
16         if serializer.is_valid():
17             user = serializer.save()
18             token, _ = Token.objects.get_or_create(user=user)
19             return Response({
20                 'message': 'User registered successfully',
21                 'token': token.key,
22                 'user': UserSerializer(user).data
23             }, status=status.HTTP_201_CREATED)
24         return Response(serializer.errors, status=status.
   HTTP_400_BAD_REQUEST)
25
26
27 class LoginView(APIView):
28     permission_classes = [AllowAny]
29
30     def post(self, request):
31         username = request.data.get('username')
32         password = request.data.get('password')
33         user = authenticate(username=username, password=password)
34         if user:
35             token, _ = Token.objects.get_or_create(user=user)
36             return Response({
37                 'token': token.key,
38                 'user': UserSerializer(user).data
39             })
40         return Response({'error': 'Invalid credentials'}, status=
   status.HTTP_401_UNAUTHORIZED)
41
42
43 class StartResearchView(APIView):
44     permission_classes = [IsAuthenticated]
45
46     def post(self, request):
```

```
47     topic = request.data.get('topic')
48     if not topic:
49         return Response({'error': 'Topic is required'}, status=
50             status.HTTP_400_BAD_REQUEST)
51
52     data = ResearchOrchestratorService.execute_research_task(
53         topic=topic, user=request.user)
54     return Response(data, status=status.HTTP_200_OK)
55
56 class ResearchHistoryView(APIView):
57     permission_classes = [IsAuthenticated]
58
59     def get(self, request):
60         tasks = ResearchTask.objects.filter(user=request.user)
61         serializer = ResearchTaskSerializer(tasks, many=True)
62         return Response(serializer.data, status=status.HTTP_200_OK)
```

Listing 4: backend/api/views.py — DRF Endpoint Handlers

18.6 11.6 Docker Containerization & Deployment

Production deployment ke liye Multi-container Docker compose orchestration set up ki gayi hai.

Listing 5: docker-compose.yml — Production Orchestration

```
version: '3.8'

services:
  django_backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    ports:
      - "8080:8080"
    environment:
      - PORT=8080
      - GROQ_API_KEY=${GROQ_API_KEY}
      - TAVILY_API_KEY=${TAVILY_API_KEY}
    volumes:
      - ./backend/db.sqlite3:/app/backend/db.sqlite3
      - ./chroma_db:/app/chroma_db
    restart: always

  streamlit_frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    ports:
      - "8501:8501"
    environment:
      - BACKEND_API_URL=http://django_backend:8080/api/v1
    depends_on:
      - django_backend
    restart: always
```

18.7 11.7 Backend & Django DRF Interview Questions (Q51 – Q65)

💡 Yaad Rakhne Ka Tip

Question 51: Tumne Streamlit Direct execution ke bajaye Django DRF backend kyun introduce kiya?

Answer: Single-tier Streamlit apps me user authentication, persistent task history, user isolation, aur API third-party integration impossible ho jata hai. Django DRF se humein Token Authentication, ORM-managed Database persistence, Service Layer separation, aur Port 8080 REST endpoints mile jinhein external clients (web, mobile, CLI) consume kar sakte hain.

💡 Yaad Rakhne Ka Tip

Question 52: Service Layer Pattern (`services.py`) ka kya benefit hai?

Answer: Agar hum LangGraph execution logic `views.py` me likhte to views heavy aur un-testable ho jate (Fat Views antipattern). Service Layer Abstraction business logic ko HTTP request handling se decouple karta hai, jisse same logic CLI, Celery tasks, ya DRF views me reuse ki ja sakti hai.

💡 Yaad Rakhne Ka Tip

Question 53: Port 8080 Collision Resolution kaise handle hua?

Answer: Default Port 8000 par machine me pehle se E-Commerce DRF server running tha. Direct call karne par 404 Page Not Found aata tha. Humne Orchestrator Backend ko explicitly Port 8080 par configure kiya aur Streamlit frontend me API base URL update kiya.

💡 Yaad Rakhne Ka Tip

Question 54: DRF me TokenAuthentication kaise kaam karti hai?

Answer: Client POST `/auth/login/` par credentials bhejta hai. Server DB me User ka unique 40-character hex key Token return karta hai. Next requests me client Authorization: `Token <key>` header bhejta hai, jise DRF ka `IsAuthenticated` permission class validate karke `request.user` set karta hai.

💡 Yaad Rakhne Ka Tip

Question 55: Multi-tenant Data Security / User Isolation kaise achieve kiya?

Answer: `Models` me `user = models.ForeignKey(User, on_delete=models.CASCADE)` enforce kiya aur har History endpoint me `QuerySet` ko strictly `ResearchTask.objects.filter(user=request.user)` se evaluate kiya, taaki user A kabhi user B ka data na dekh sake.